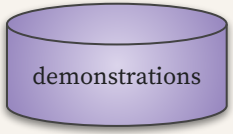
Лекция 9: Демонстрации и трансформеры

План лекции (начало):

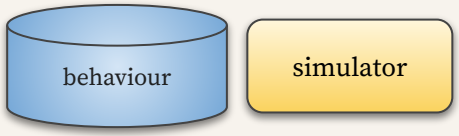
- **BC:** Behavioral Cloning
- **GATO:** A Generalist Agent [link]
- **MAPF-GPT:** BC + multi-agent [link]

- **Dagger:** A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning [link]

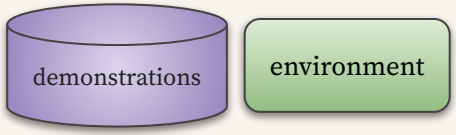


- **CQL:** Conservative Q-Learning for Offline Reinforcement Learning [link]
- **Decision Transformer:** Decision Transformer: Reinforcement Learning via Sequence Modeling [link]
- **Trajectory Transformer:** Offline Reinforcement Learning as One Big Sequence Modeling Problem [link]
- **MTM:** Masked Trajectory Models for Prediction, Representation, and Control [link]
- **TAP:** Efficient Planning in a Compact Latent Action Space [link]
- **Diffuser:** Planning with Diffusion for Flexible Behavior Synthesis [link]

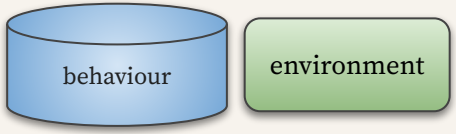
План лекции (продолжение):



- **SQIL:** Imitation Learning via Reinforcement Learning with Sparse Rewards [link]
- **GAIL:** Generative Adversarial Imitation Learning [link]



- **DQfD:** Deep Q-learning From Demonstrations [link]
- **R2D3:** Making Efficient Use of Demonstrations to Solve Hard Exploration Problems [link]
- **ForgER:** Forgetful experience replay in hierarchical reinforcement learning from expert demonstrations [link]
- **SMART**: Self-supervised Multi-task Pretraining with Control Transformers [link]

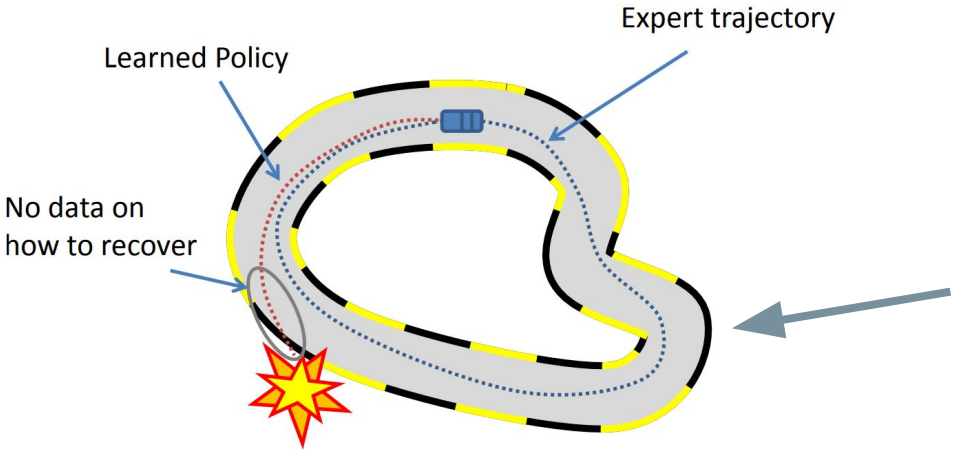


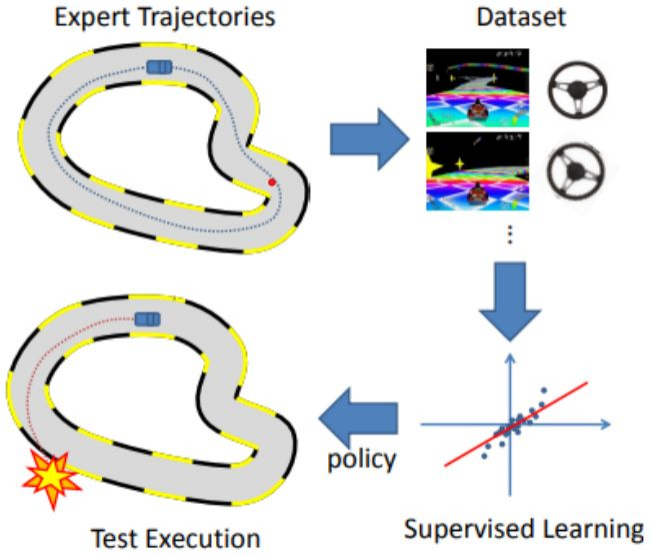
- **VPT:** Video PreTraining (VPT): Learning to Act by Watching Unlabeled Online Videos [link]
- **AlphaStar:** Grandmaster level in StarCraft II using multi-agent reinforcement learning [link]

Обучение по экспертному поведению

- **BC:** Behavioral Cloning
- **GATO:** A Generalist Agent

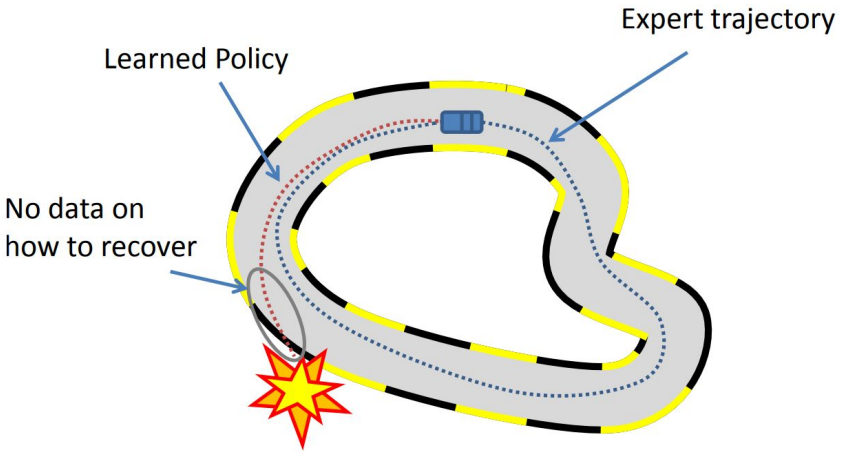
Копирование поведения:





$$F(\theta) = \sum_{i=1}^{n} \ln f_i(y_i \mid \theta).$$

Копирование поведения:



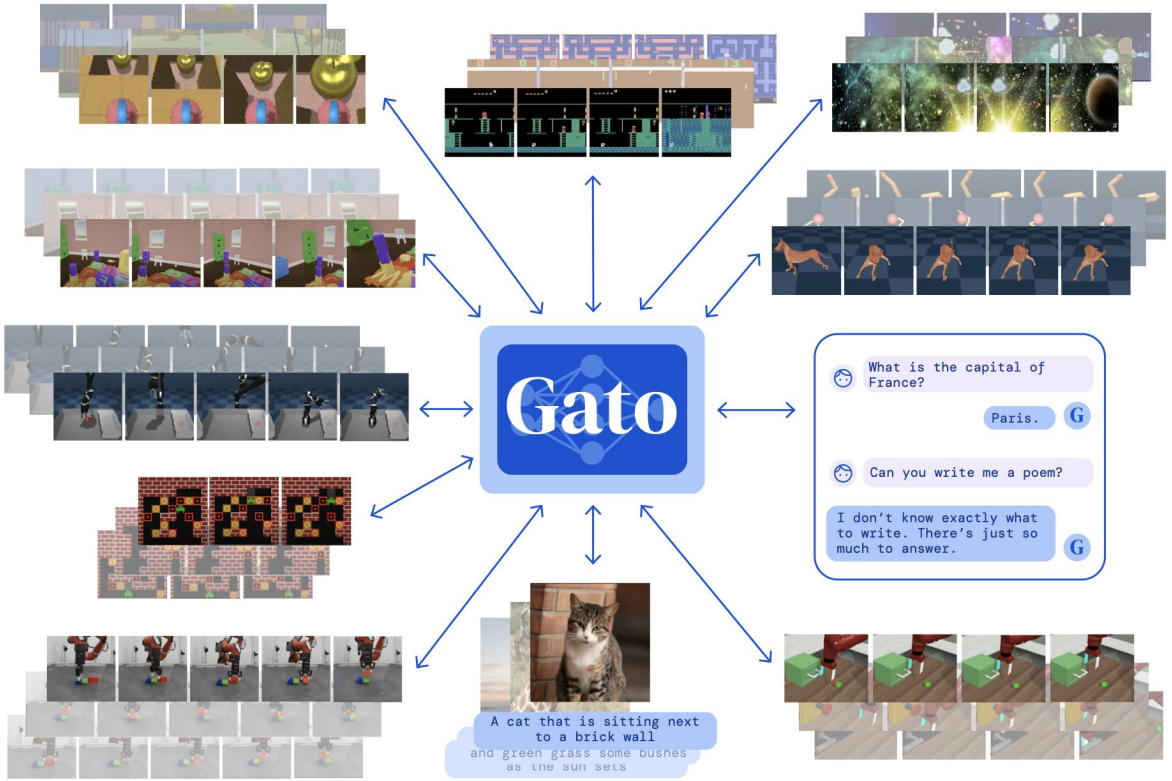
$E[\text{Total errors}] \leq \varepsilon(T + (T-1) + (T-2) + \ldots + 1) \propto \varepsilon T^2$

Пример 1: A Generalist Agent, GATO:

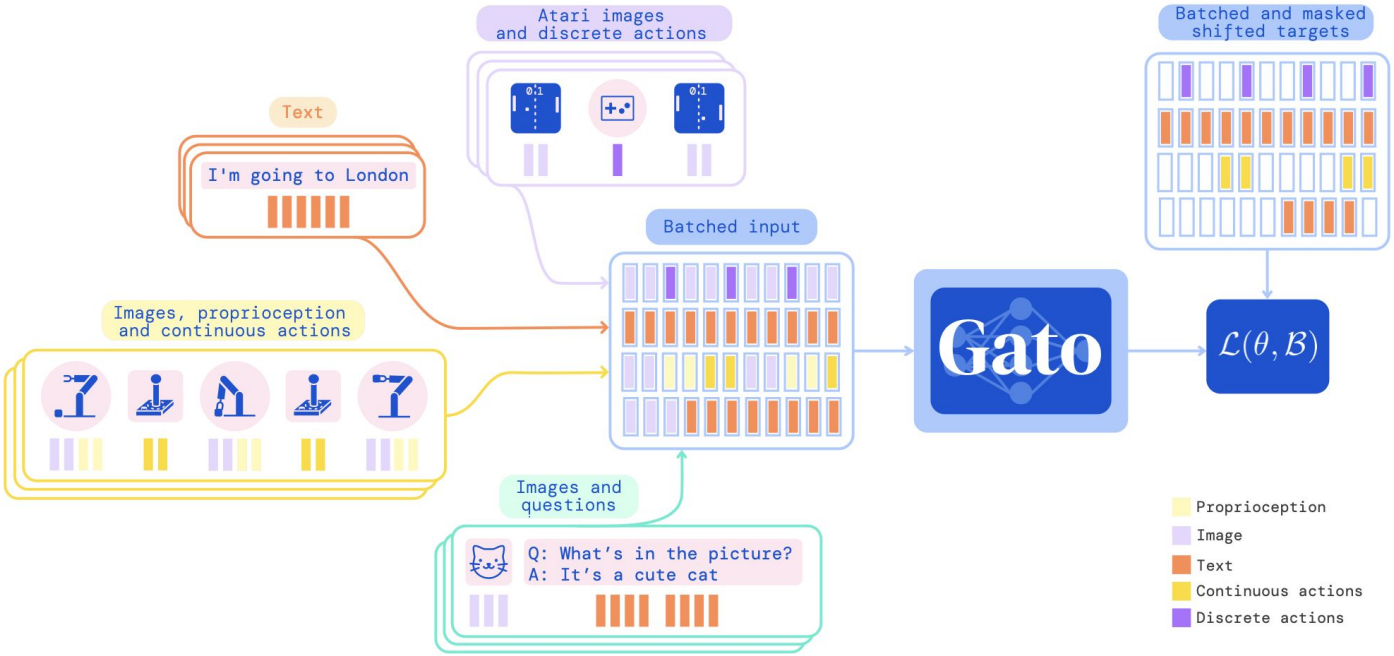
GATO продемонстрировал результаты в более чем 600 задачах, включая: видеоигры, image captioning, общение в чате и управление роботизированной рукой.

GATO работает по простому принципу: он принимает сенсорные входные данные (например, изображения или текст), выдает действия.

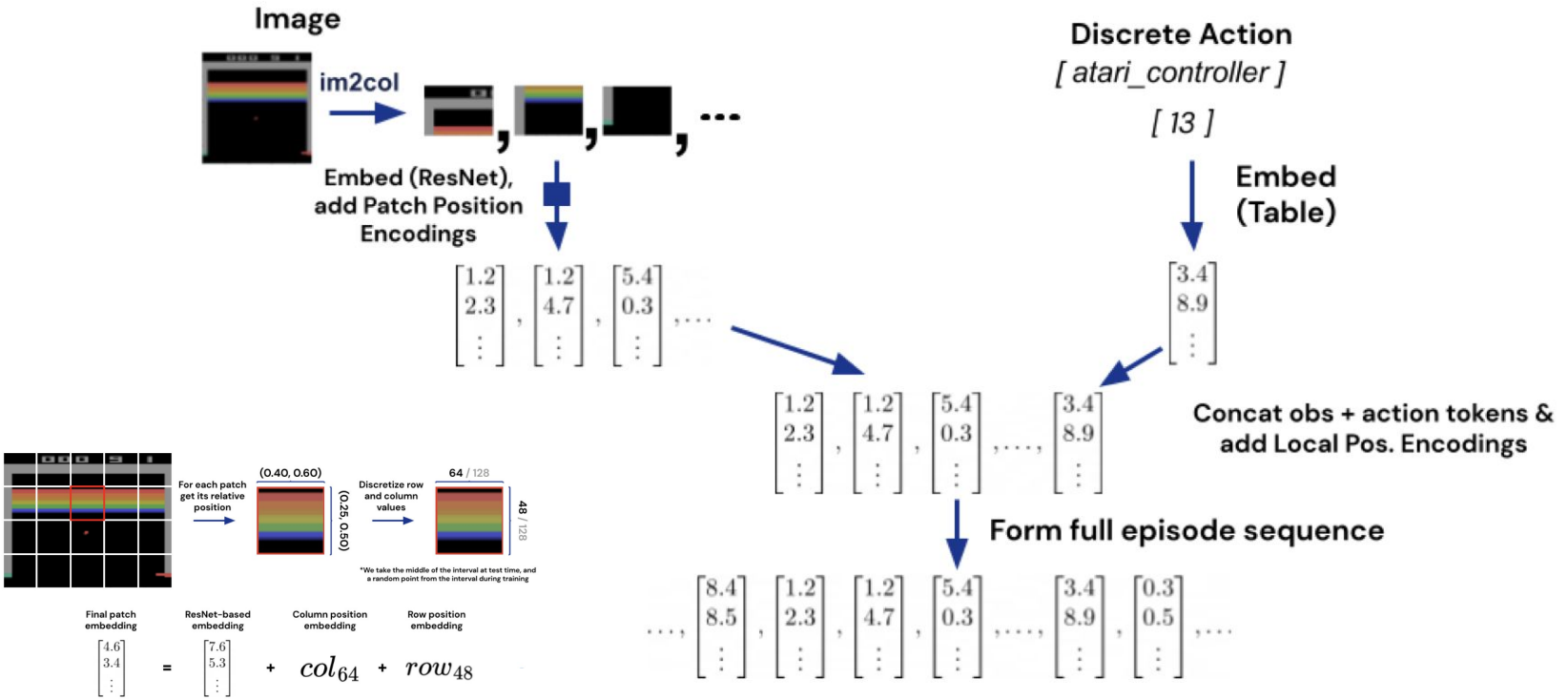
Используется масштабная нейронная сеть на основе трансформера, аналогичную моделям NLP.



Мультимодальный вход:



Кодирование наблюдений:



Функция потерь:

$$\mathcal{L}(\theta, \mathcal{B}) = -\sum_{b=1}^{|\mathcal{B}|}\sum_{l=1}^{L} m\,(b,l)\log p_\theta\left(s_l^{(b)}|s_1^{(b)},\ldots,s_{l-1}^{(b)}\right)$$

Функция потерь:

$$\mathcal{L}(\theta, \mathcal{B}) = -\sum_{b=1}^{|\mathcal{B}|}\sum_{l=1}^{L} m\,(b,l)\log p_\theta\left(s_l^{(b)}|s_1^{(b)},\ldots,s_{l-1}^{(b)}\right)$$

условная вероятность следующего элемента

Функция потерь:

$$\mathcal{L}(\theta, \mathcal{B}) = -\sum_{b=1}^{|\mathcal{B}|}\sum_{l=1}^{L} m\,(b, l) \log p_\theta \left(s_l^{(b)} | s_1^{(b)}, \ldots, s_{l-1}^{(b)} \right)$$

логарифмическая функция правдоподобия

Функция потерь:

функция маскирования
1 если действие/токен

$$\mathcal{L}(\theta, \mathcal{B}) = -\sum_{b=1}^{|\mathcal{B}|} \sum_{l=1}^{L} m\,(b, l) \log p_\theta \left(s_l^{(b)} | s_1^{(b)}, \ldots, s_{l-1}^{(b)} \right)$$

Функция потерь:

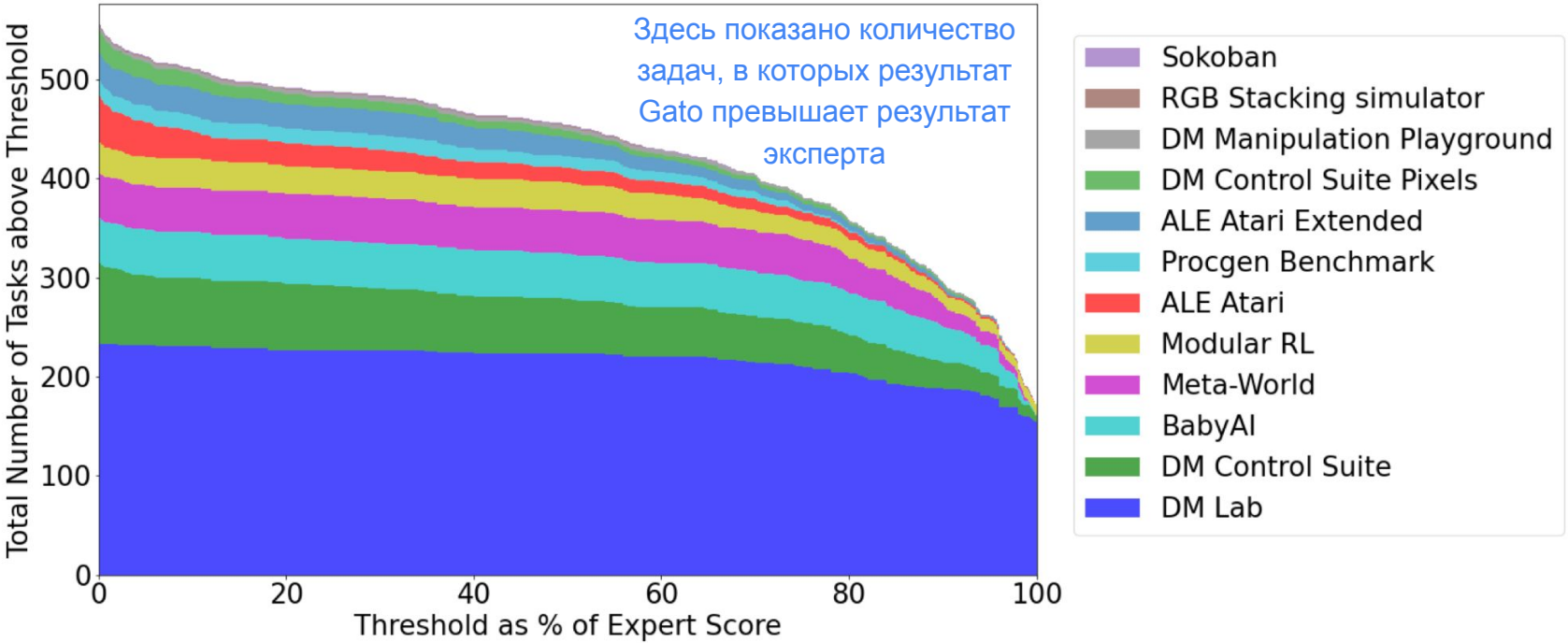
элемент траектории

$$\mathcal{L}(\theta, \mathcal{B}) = -\sum_{b=1}^{|\mathcal{B}|} \sum_{l=1}^{L} m\,(b, l) \log p_\theta \left(s_l^{(b)} | s_1^{(b)}, \ldots, s_{l-1}^{(b)} \right)$$

Функция потерь:

$$\mathcal{L}(\theta, \mathcal{B}) = -\overbrace{\sum_{b=1}^{|\mathcal{B}|}}^{\text{батч}} \sum_{l=1}^{L} m\,(b, l) \log p_\theta \left(s_l^{(b)} | s_1^{(b)}, \ldots, s_{l-1}^{(b)} \right)$$

Результаты:


Здесь показано количество задач, в которых результат Gato превышает результат эксперта
Total Number of Tasks above Threshold
0
100
200
300
400
500
0
20
40
60
80
100
Threshold as % of Expert Score
Sokoban
RGB Stacking simulator
DM Manipulation Playground
DM Control Suite Pixels
ALE Atari Extended
Procgen Benchmark
ALE Atari
Modular RL
Meta-World
BabyAI
DM Control Suite
DM Lab

Визуализация эмбеддингов:

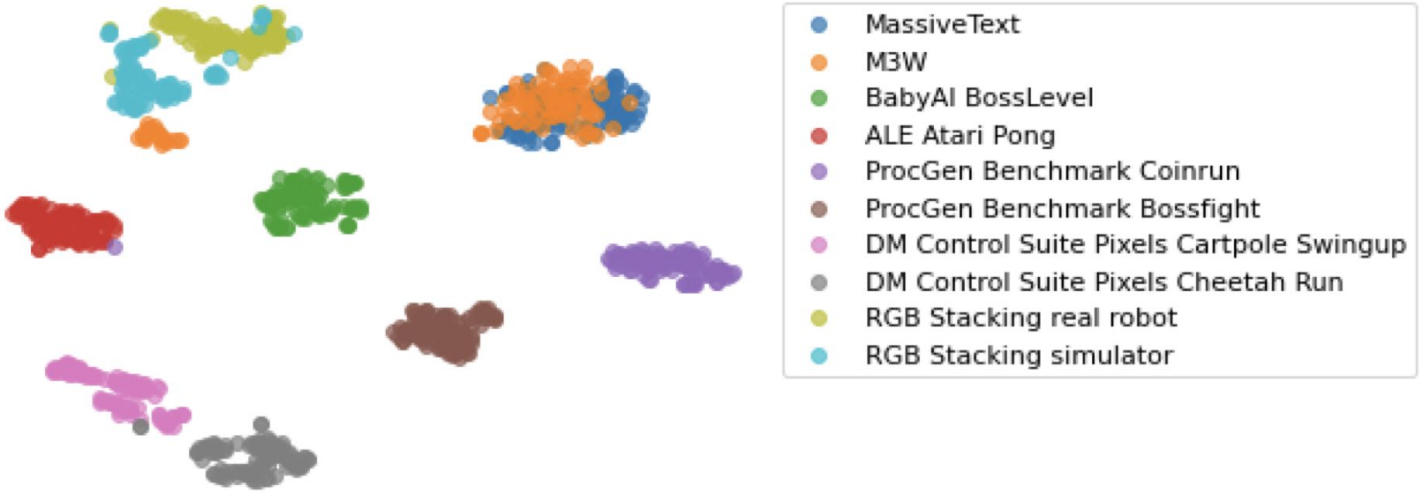
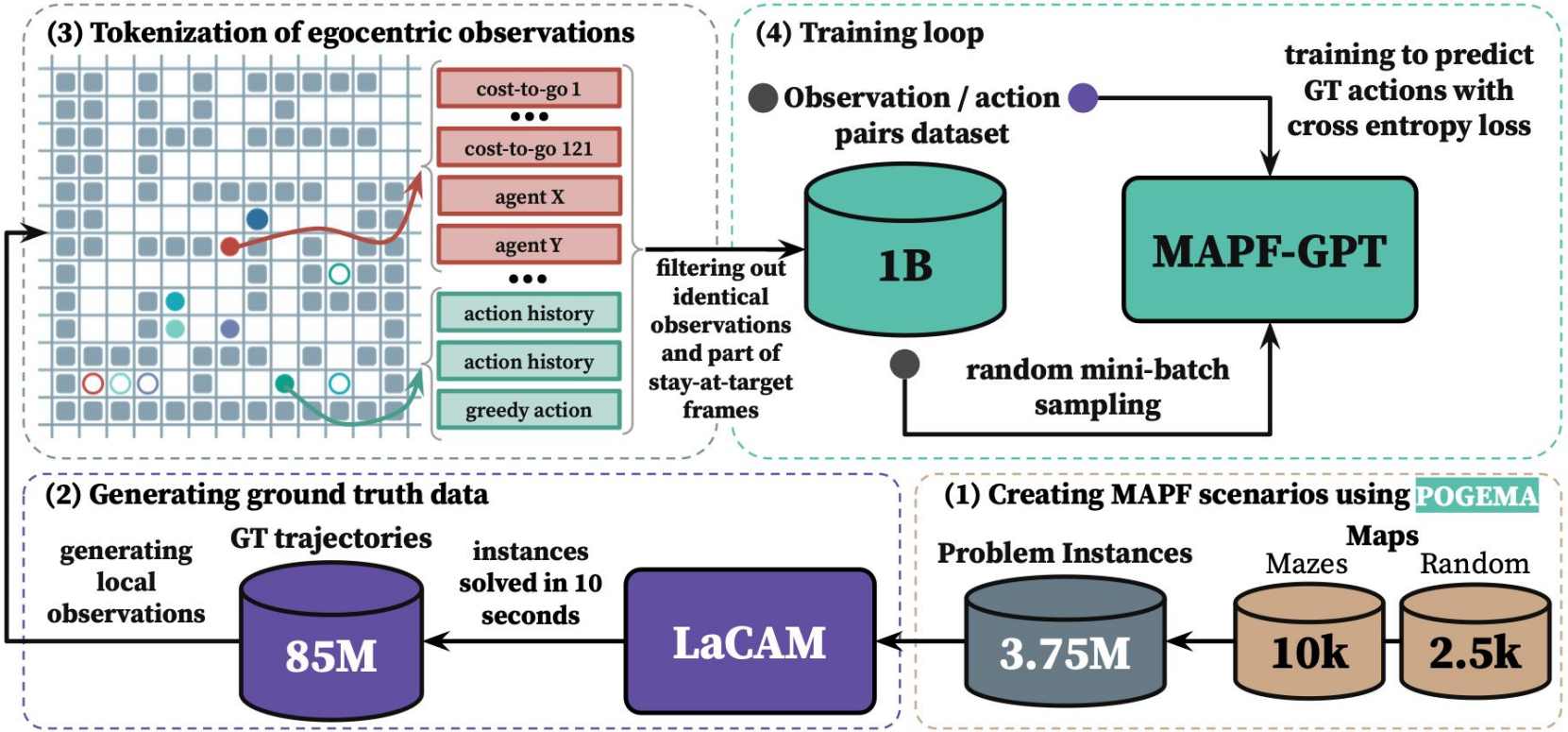


Figure 13: **Embedding visualization.** T-SNE visualization of embeddings from different tasks. A large part of the vision-language embeddings (M3W) overlaps with the language cluster (MassiveText). Other tasks involving actions fall in their own cluster.

Пример 2: MAPF-GPT



Пример 2: Результаты

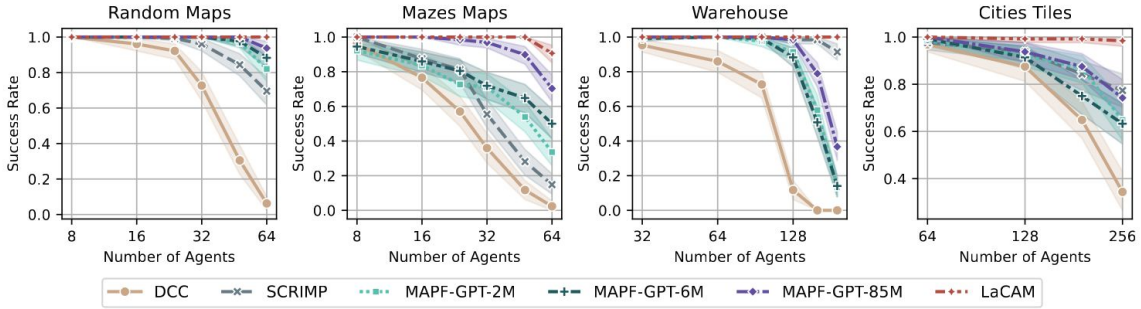


Figure 3: Success rate of the evaluated MAPF solvers on different maps. The shaded area indicates 95% confidence intervals.

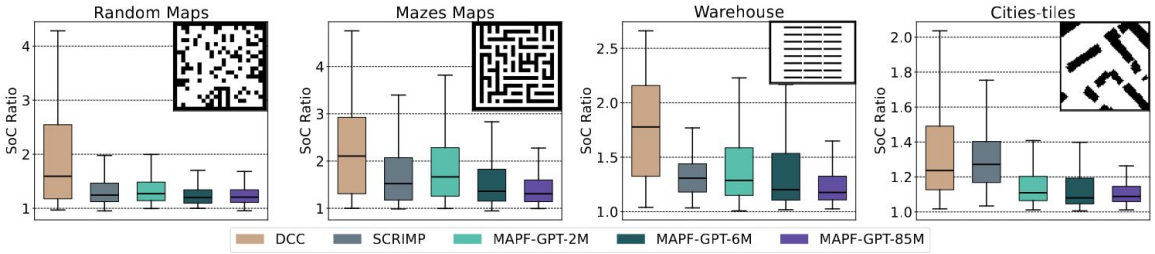
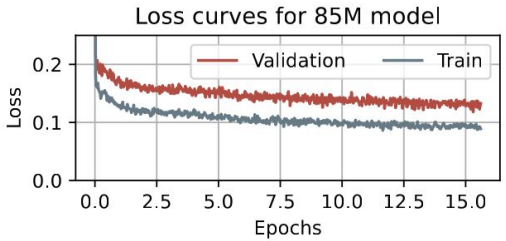


Figure 4: Quality of the obtained solutions relative to the ones of LaCAM (lower is better).



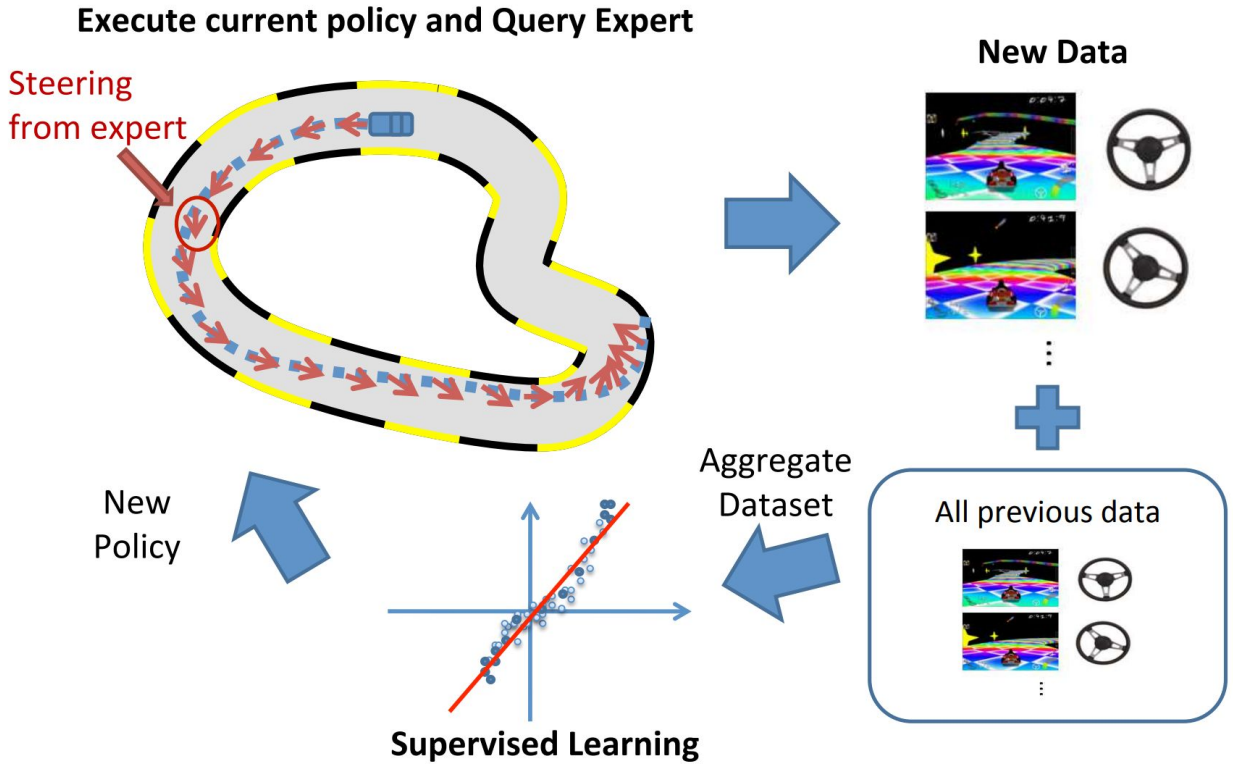
Parameter	2M	6M	85M
Number of epochs	46.875	12.5	15.625
Total training iterations	15,000	30,000	1,000,000
Batch size	4096	2048	512
Number of layers	5	8	12
Number of attention heads	5	8	12
Embedding size	160	256	768

Table 4: Model-specific hyperparameters.

Обучение по эксперту и симулятору

- **Dagger:** A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning

Dagger:


Execute current policy and Query Expert
Steering
from expert
New Data
Aggregate
Dataset
All previous data
New
Policy
Supervised Learning

Dagger:

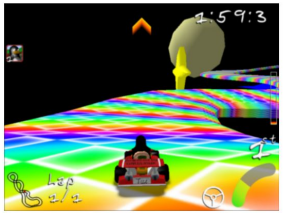


Figure 1: Image from Super Tux Kart's Star Track.

Figure 3: Captured image from Super Mario Bros.

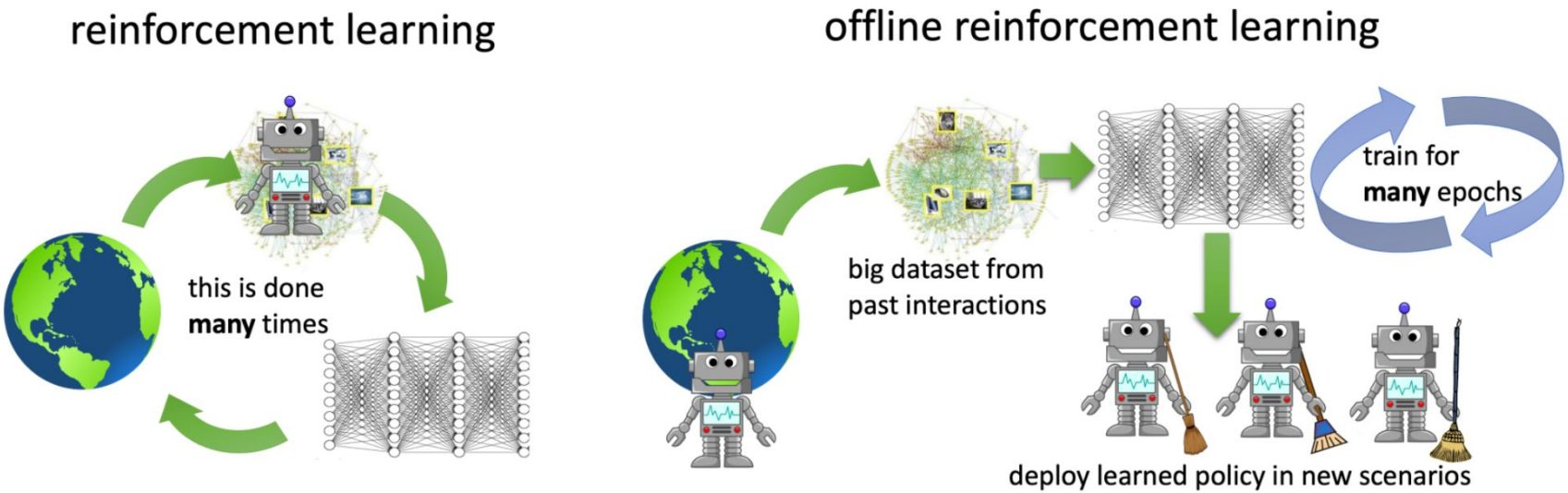
Initialize $\mathcal{D} \leftarrow \emptyset$.
Initialize $\hat{\pi}_1$ to any policy in Π.
for $i = 1$ **to** N **do**
  Let $\pi_i = \beta_i \pi^* + (1 - \beta_i)\hat{\pi}_i$.
  Sample T-step trajectories using π_i.
  Get dataset $\mathcal{D}_i = \{(s, \pi^*(s))\}$ of visited states by π_i and actions given by expert.
  Aggregate datasets: $\mathcal{D} \leftarrow \mathcal{D} \bigcup \mathcal{D}_i$.
  Train classifier $\hat{\pi}_{i+1}$ on $\mathcal{D}$ (or use online learner to get $\hat{\pi}_{i+1}$ given new data $\mathcal{D}_i$).
end for
Return best $\hat{\pi}_i$ on validation.

Смешанная стратегия

Обучение по демонстрациям

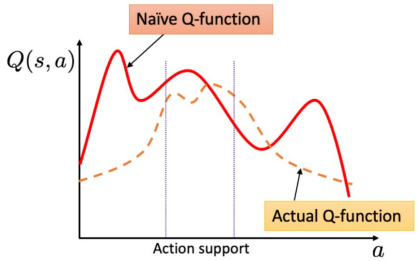
- **CQL:** Conservative Q-Learning for Offline Reinforcement Learning
- **Decision Transformer:** Decision Transformer: Reinforcement Learning via Sequence Modeling
- **Trajectory Transformer:** Offline Reinforcement Learning as One Big Sequence Modeling Problem
- **Diffuser:** Planning with Diffusion for Flexible Behavior Synthesis
- **MTM:** Masked Trajectory Models for Prediction, Representation, and Control
- **TAP:** Efficient Planning in a Compact Latent Action Space

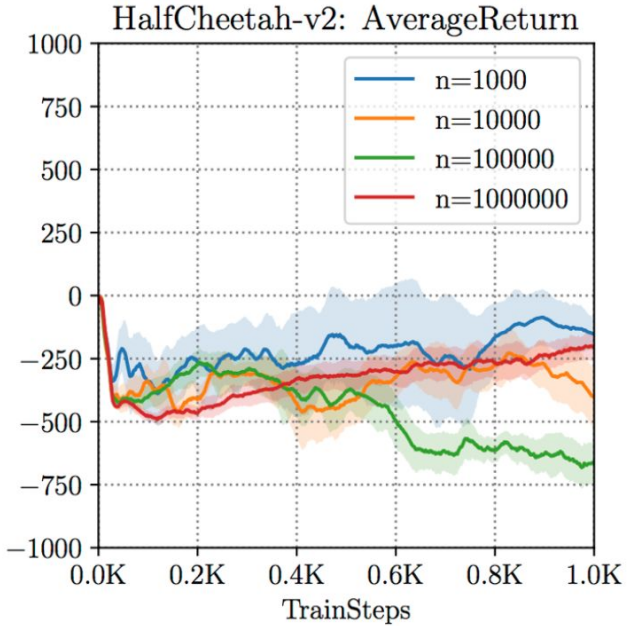
Offline RL:



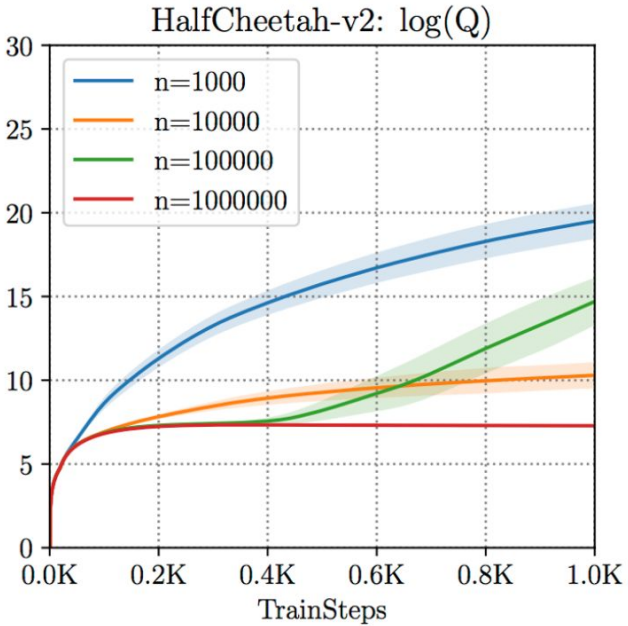
Offline RL

Conservative Q-Learning for Offline RL





How well it does



How well **it thinks** it does

(Kumar et al., 2020)

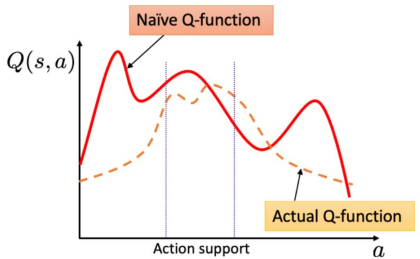
Conservative Q-Learning for Offline Reinforcement Learning:

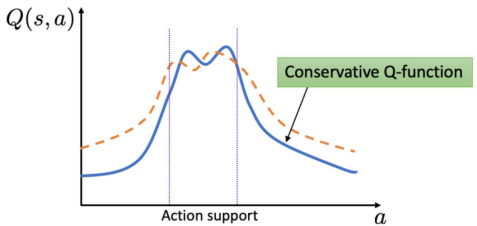
To obtain this lower-bound on the actual Q-value function of the policy, CQL trains the Q-function using a sum of two objectives — standard TD error and a regularizer that minimizes Q-values on unseen actions with overestimated values while simultaneously maximizing the expected Q-value on the dataset:

$$\hat{Q}^{\pi}_{\text{CQL}} \leftarrow \arg\min_{Q} \max_{\mu(a|s)} \underbrace{\left(\mathbb{E}_{s\sim\text{data},a\sim\mu}[Q(s,a)] - \mathbb{E}_{s,a\sim\text{data}}[Q(s,a)]\right)}_{\text{CQL regularizer}} + \frac{1}{2\alpha} \underbrace{\mathbb{E}_{s,a,s'\sim\text{data}}\left[\left(r(s,a) + \gamma\mathbb{E}_{\pi}[\bar{Q}(s',a')] - Q(s,a)\right)^2\right]}_{\text{standard TD error}}$$

Algorithm 1 Conservative Q-Learning (both variants)

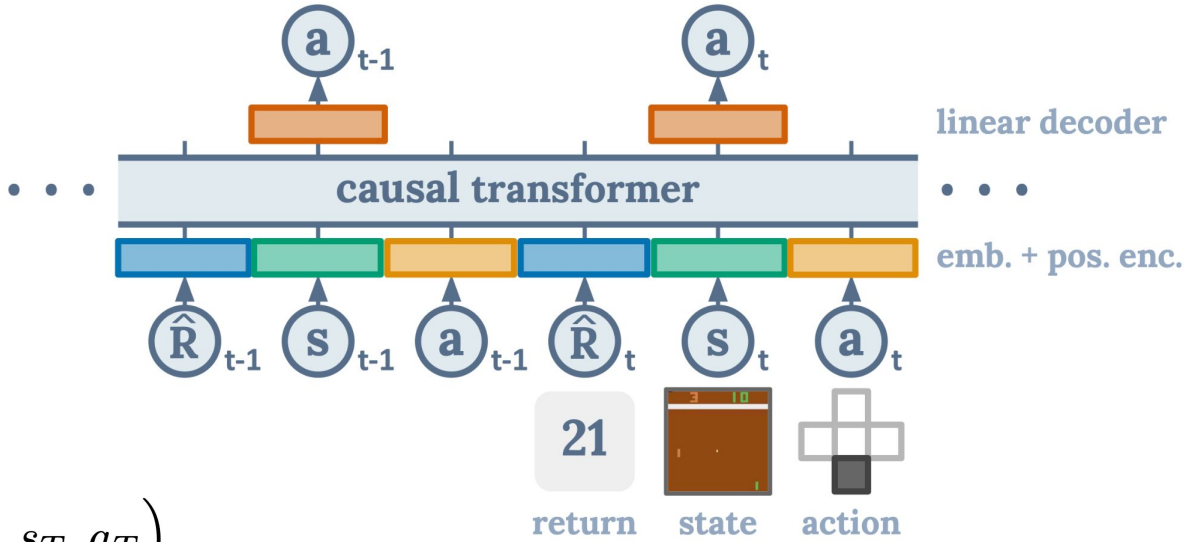
1: Initialize Q-function, Q_θ, and optionally a policy, π_ϕ.
2: **for** step t in $\{1, \ldots, \text{N}\}$ **do**
3: Train the Q-function using G_Q gradient steps on objective from Equation 4
$\theta_t := \theta_{t-1} - \eta_Q \nabla_\theta \text{CQL}(\mathcal{R})(\theta)$
(Use $\mathcal{B}^*$ for Q-learning, $\mathcal{B}^{\pi_{\phi_t}}$ for actor-critic)
4: (only with actor-critic) Improve policy π_ϕ via G_π gradient steps on ϕ with SAC-style entropy regularization:
$\phi_t := \phi_{t-1} + \eta_\pi \mathbb{E}_{\mathbf{s}\sim\mathcal{D},\mathbf{a}\sim\pi_\phi(\cdot|\mathbf{s})}[Q_\theta(\mathbf{s},\mathbf{a}) - \log \pi_\phi(\mathbf{a}|\mathbf{s})]$
5: **end for**





Decision Transformer:

- GPT-like архитектура
- **Cross entropy loss** для дискретного пространства действие, **MSE** для непрерывного
- Помимо состояний и действия добавляется **суммарное вознаграждение**



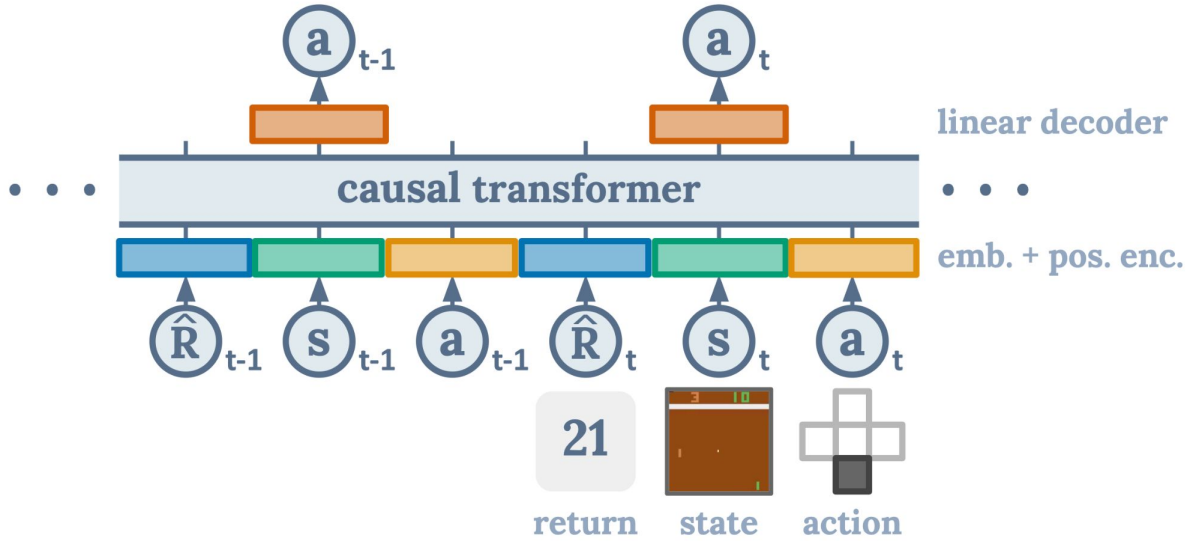
$$\tau = \left(\widehat{R}_1, s_1, a_1, \widehat{R}_2, s_2, a_2, \ldots, \widehat{R}_T, s_T, a_T\right)$$

$$\widehat{R}_t = \sum_{t'=t}^{T} r_{t'}$$

суммарное вознаграждение, без дисконтирования! (return-to-go)

Decision Transformer:

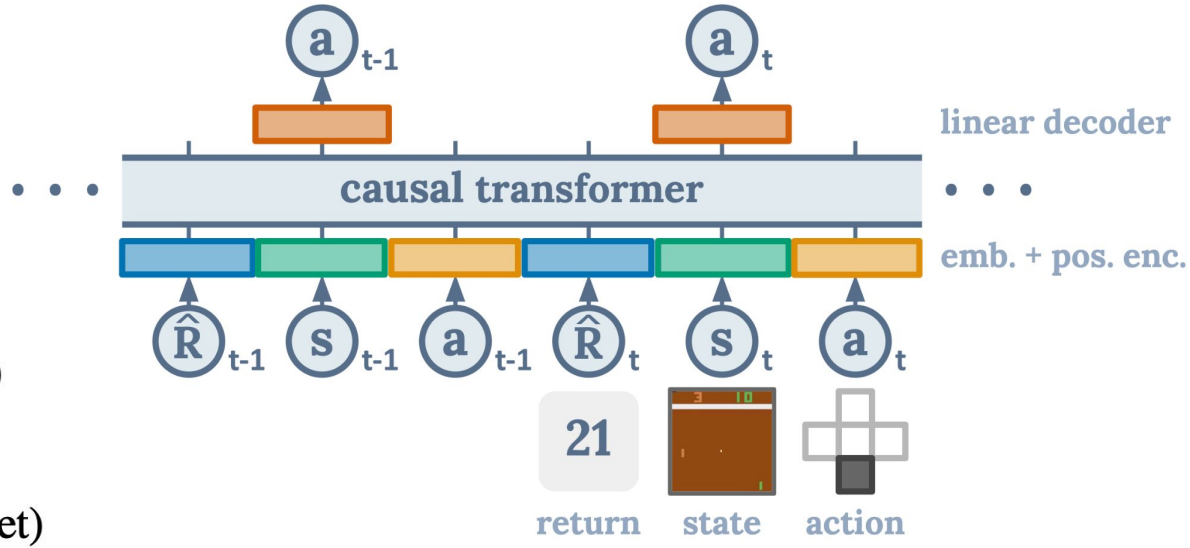
Как заставить такую модель генерировать самые выгодные действия?



Decision Transformer:

Возьмем максимальные значения из лучших траекторий, которые есть в датасете! (return-to-go conditioning)

90 Breakout ($\approx 1\times$ max in dataset)
2500 Qbert ($\approx 5\times$ max in dataset)
20 Pong ($\approx 1\times$ max in dataset)
1450 Seaquest ($\approx 5\times$ max in dataset)



Результаты на OfflineRL датасетах:

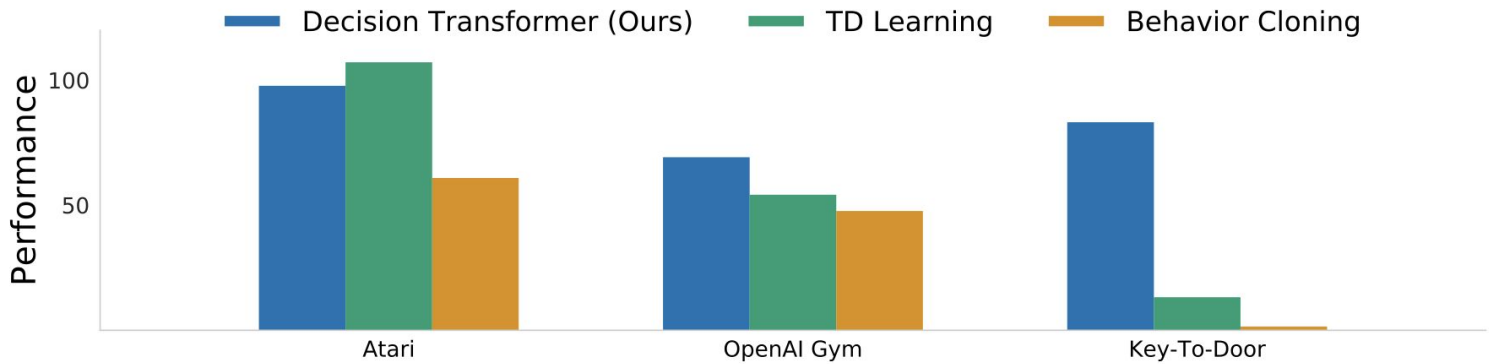


Figure 3: Results comparing Decision Transformer (ours) to TD learning (CQL) and behavior cloning across Atari, OpenAI Gym, and Minigrid. On a diverse set of tasks, Decision Transformer performs comparably or better than traditional approaches. Performance is measured by normalized episode return (see text for details).

Результаты на OfflineRL датасетах:

Game	DT (Ours)	CQL	QR-DQN	REM	BC
Breakout	$\mathbf{267.5 \pm 97.5}$	211.1	17.1	8.9	138.9 ± 61.7
Qbert	15.4 ± 11.4	**104.2**	0.0	0.0	17.3 ± 14.7
Pong	106.1 ± 8.1	**111.9**	18.0	0.5	85.2 ± 20.0
Seaquest	$\mathbf{2.5 \pm 0.4}$	1.7	0.4	0.7	2.1 ± 0.3

Table 1: Gamer-normalized scores for the 1% DQN-replay Atari dataset. We report the mean and variance across 3 seeds. Best mean scores are highlighted in bold. Decision Transformer (DT) performs comparably to CQL on 3 out of 4 games, and outperforms other baselines in most games.

Результаты на OfflineRL датасетах:

Dataset	Environment	DT (Ours)	CQL	BEAR	BRAC-v	AWR	BC
Medium-Expert	HalfCheetah	**86.8 ± 1.3**	62.4	53.4	41.9	52.7	59.9
Medium-Expert	Hopper	107.6 ± 1.8	**111.0**	96.3	0.8	27.1	79.6
Medium-Expert	Walker	**108.1 ± 0.2**	98.7	40.1	81.6	53.8	36.6
Medium-Expert	Reacher	**89.1 ± 1.3**	30.6	-	-	-	73.3
Medium	HalfCheetah	42.6 ± 0.1	44.4	41.7	**46.3**	37.4	43.1
Medium	Hopper	**67.6 ± 1.0**	58.0	52.1	31.1	35.9	63.9
Medium	Walker	74.0 ± 1.4	79.2	59.1	**81.1**	17.4	77.3
Medium	Reacher	**51.2 ± 3.4**	26.0	-	-	-	**48.9**
Medium-Replay	HalfCheetah	36.6 ± 0.8	46.2	38.6	**47.7**	40.3	4.3
Medium-Replay	Hopper	**82.7 ± 7.0**	48.6	33.7	0.6	28.4	27.6
Medium-Replay	Walker	**66.6 ± 3.0**	26.7	19.2	0.9	15.5	36.9
Medium-Replay	Reacher	**18.0 ± 2.4**	**19.0**	-	-	-	5.4
Average (Without Reacher)		**74.7**	63.9	48.2	36.9	34.3	46.4
Average (All Settings)		**69.2**	54.2	-	-	-	47.7

Table 2: Results for D4RL datasets[3]. We report the mean and variance for three seeds. Decision Transformer (DT) outperforms conventional RL algorithms on almost all tasks.

Рекомендации по применению DT, CQL, BC

Выбор агента в зависимости от характеристик вознаграждения:

- CQL лучше подходит для сред с плотным вознаграждением, но его эффективность может колебаться в других условиях.
- DT выделяется в средах с разреженным вознаграждением.

Эффективность использования данных:

- CQL является наиболее эффективным при наличии ограниченного количества высококачественных данных.
- DT требует больше данных, но лучше масштабируется с увеличением объема субоптимальных данных.

Устойчивость к качеству данных:

- BC предпочтителен, когда данные генерируются людьми и известно, что они высококачественные.
- DT и CQL более устойчивы к шумным данным, при этом DT обычно показывает более надежные результаты.

Влияние увеличения размерности и горизонта задач:

- Все агенты испытывают ухудшение при увеличении размерности пространства состояний.
- При увеличении горизонта задач DT остается надежным выбором, а BC может быть предпочтительнее при высококачественных данных.

https://openreview.net/forum?id=vpV7fOFQy4

Trajectory Transformer:

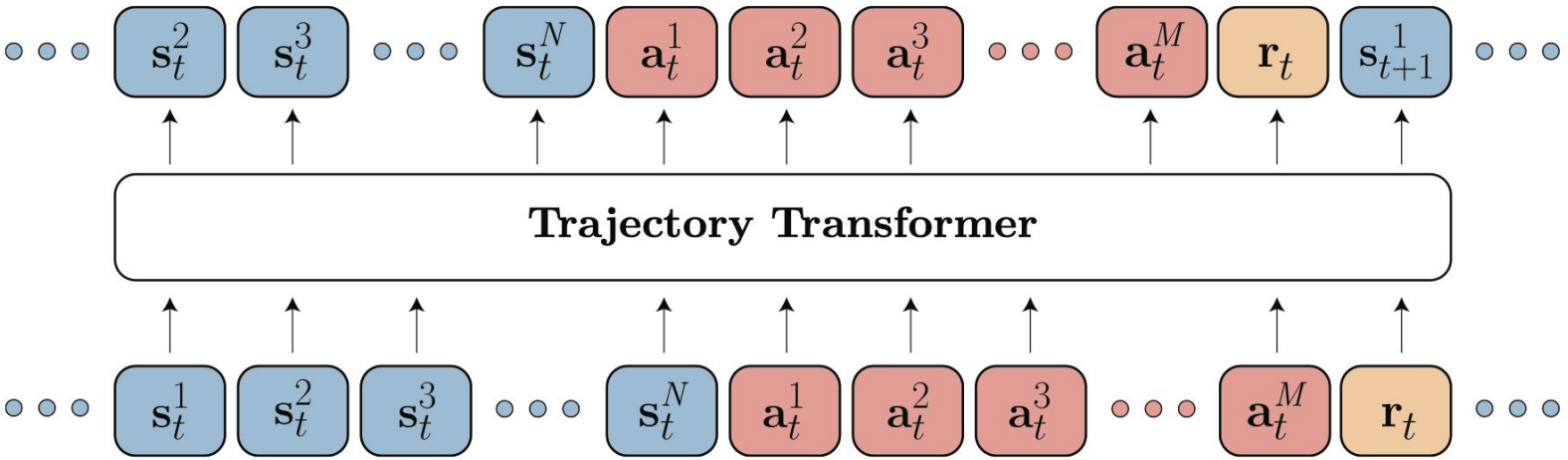
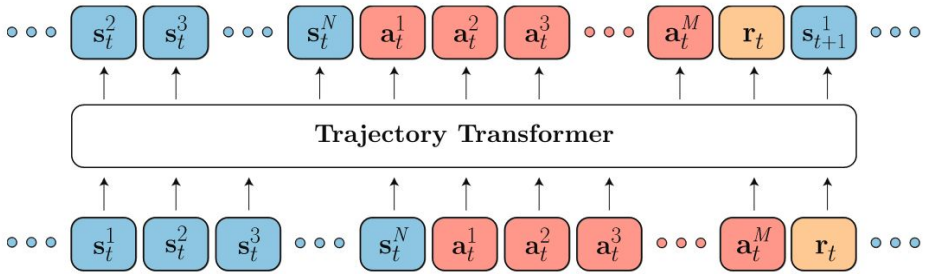


Figure 1 (Architecture) The Trajectory Transformer trains on sequences of (autoregressively discretized) states, actions, and rewards. Planning with the Trajectory Transformer mirrors the sampling procedure used to generate sequences from a language model.

Trajectory Transformer:

- при обучении используются состояния, действия и вознаграждения
- каузальное предсказание, используя будущие состояния, действия и вознаграждения (goal-based), с выборочным маскированием
- использует beam search (лучевой поиск) при сэмплировании
- по сути является методом планирования



$$\mathcal{L}(\tau) = \sum_{t=1}^{T} \Big(\underbrace{\sum_{i=1}^{N} \log P_\theta\big(s_t^i \,|\, \mathbf{s}_t^{<i}, \boldsymbol{\tau}_{<t}\big)}_{\text{состояния}} + \underbrace{\sum_{j=1}^{M} \log P_\theta\big(a_t^j \,|\, \mathbf{a}_t^{<j}, \mathbf{s}_t, \boldsymbol{\tau}_{<t}\big)}_{\text{действия}} + \underbrace{\log P_\theta\big(r_t \,|\, \mathbf{a}_t, \mathbf{s}_t, \boldsymbol{\tau}_{<t}\big)}_{\text{вознаграждения}} \Big)$$

история

Лучевой поиск:

Algorithm 1 Beam search

Require Input sequence $\mathbf{x}$, vocabulary $\mathcal{V}$, sequence length T, beam width B
Initialize $Y_0 = \{ ( ) \}$
for $t = 1, \ldots, T$ **do**
  $\mathcal{C}_t \leftarrow \{\mathbf{y}_{t-1} \circ y \mid \mathbf{y}_{t-1} \in Y_{t-1} \text{ and } y \in \mathcal{V}\}$ // candidate single-token extensions
  $Y_t \leftarrow \underset{Y \subseteq \mathcal{C}_t, \, \lvert Y \rvert = B}{\operatorname{argmax}} \log P_\theta(Y \mid \mathbf{x})$ // B most likely sequences from candidates
end for
Return $\underset{\mathbf{y} \in Y_T}{\operatorname{argmax}} \log P_\theta(\mathbf{y} \mid \mathbf{x})$

1) Расширение узлов: На каждом шаге алгоритм расширяет все текущие узлы, генерируя набор возможных последующих узлов.

2) Оценка и отбор: Каждый из этих новых узлов оценивается с использованием специальной эвристической функции. Выбирается фиксированное количество лучших узлов (например, узлов с наивысшей оценкой). Это количество узлов называется "шириной луча".

Результаты:

варианты лучевого поиска

Dataset	Environment	BC	MBOP	BRAC	CQL	DT	TT (uniform)	TT (quantile)
Med-Expert	HalfCheetah	59.9	105.9	41.9	91.6	86.8	40.8 ±2.3	95.0 ±0.2
Med-Expert	Hopper	79.6	55.1	0.9	105.4	107.6	106.0 ±0.28	110.0 ±2.7
Med-Expert	Walker2d	36.6	70.2	81.6	108.8	108.1	91.0 ±2.8	101.9 ±6.8
Medium	HalfCheetah	43.1	44.6	46.3	44.0	42.6	44.0 ±0.31	46.9 ±0.4
Medium	Hopper	63.9	48.8	31.3	58.5	67.6	67.4 ±2.9	61.1 ±3.6
Medium	Walker2d	77.3	41.0	81.1	72.5	74.0	81.3 ±2.1	79.0 ±2.8
Med-Replay	HalfCheetah	4.3	42.3	47.7	45.5	36.6	44.1 ±0.9	41.9 ±2.5
Med-Replay	Hopper	27.6	12.4	0.6	95.0	82.7	99.4 ±3.2	91.5 ±3.6
Med-Replay	Walker2d	36.9	9.7	0.9	77.2	66.6	79.4 ±3.3	82.6 ±6.9
Average		47.7	47.8	36.9	77.6	74.7	72.6	78.9

Masked Trajectory Modelling:

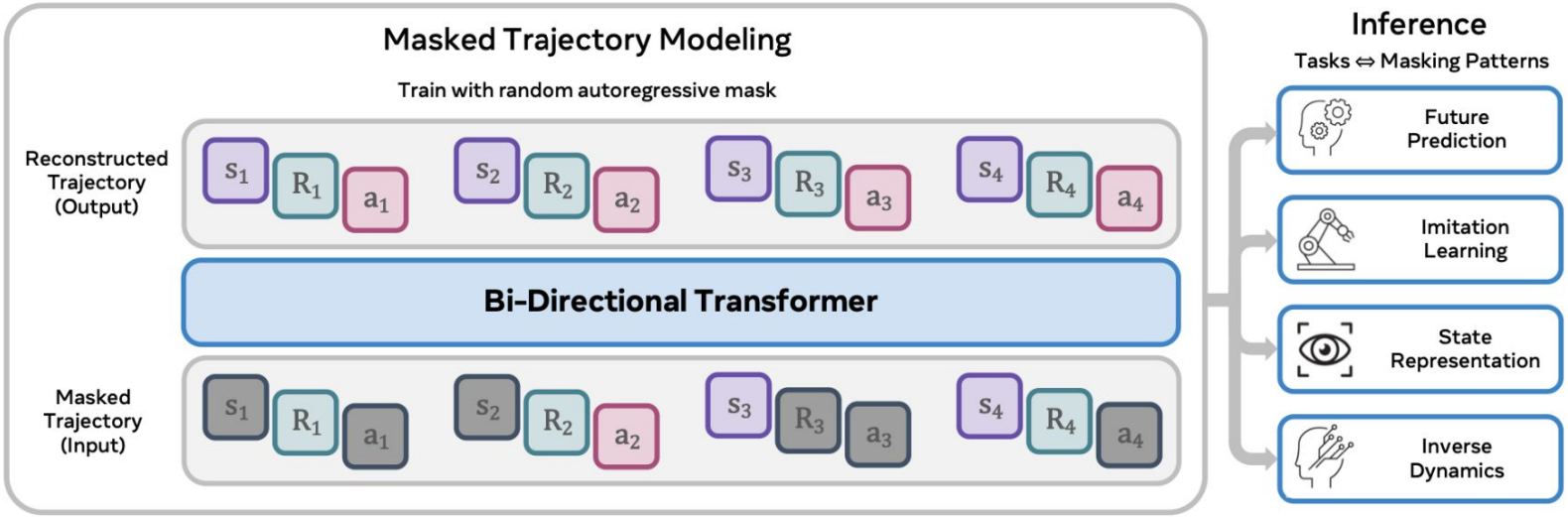


Figure 1. **Masked Trajectory Modeling (MTM) Framework.** (Left) The training process involves reconstructing trajectory segments from a randomly masked view of the same. (Right) After training, MTM can enable several downstream use-cases by simply changing the masking pattern at inference time. See Section 3 for discussion on training and inference masking patterns.

Trajectory Autoencoding Planner (TAP):

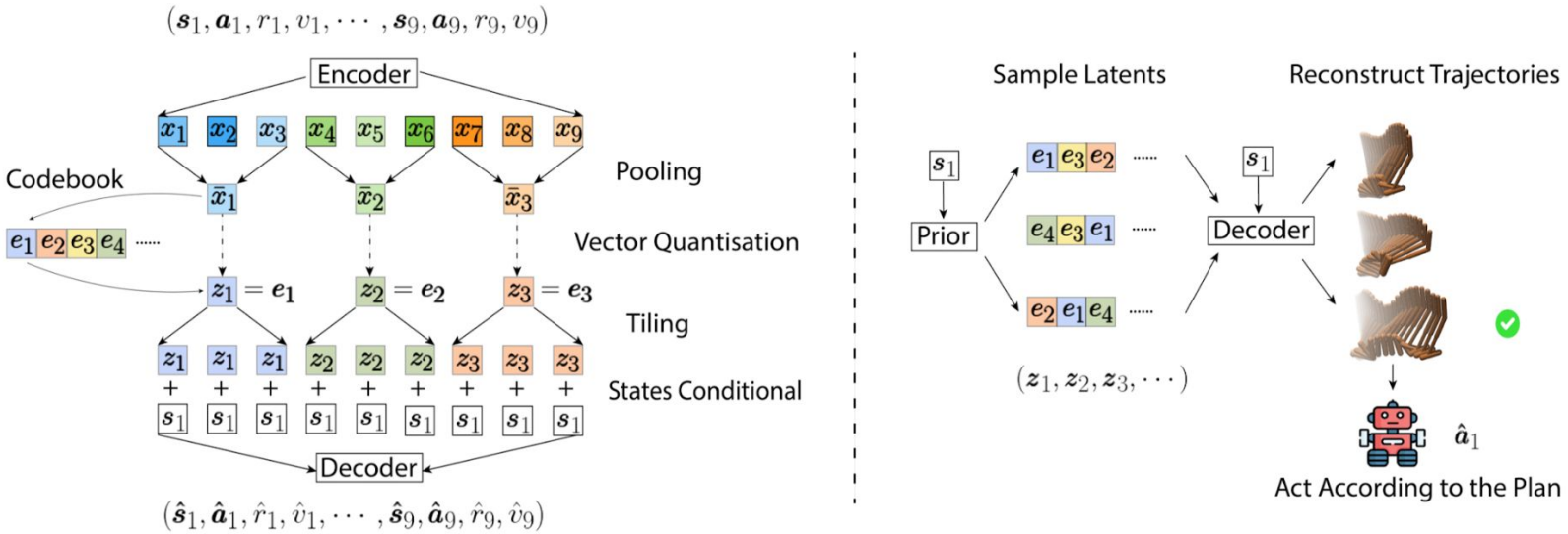


Figure 2: Illustration of the training and test time inference process of TAP. The left-hand side shows the training process, highlighting the design of the bottleneck. The right-hand side figure shows how we generate plans during the test time, with vanilla sampling.

Diffuser:

Algorithm 1 Guided Diffusion Planning

Require Diffuser μ_θ, guide $\mathcal{J}$, scale α, covariances Σ^i
while not done **do**
  Observe state $\mathbf{s}$; initialize plan $\boldsymbol{\tau}^N \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$
  for $i = N, \ldots, 1$ **do**
    // parameters of reverse transition
    $\mu \leftarrow \mu_\theta(\boldsymbol{\tau}^i)$
    // guide using gradients of return
    $\boldsymbol{\tau}^{i-1} \sim \mathcal{N}(\mu + \alpha\Sigma\nabla\mathcal{J}(\mu), \Sigma^i)$
    // constrain first state of plan
    $\boldsymbol{\tau}^{i-1}_{\mathbf{s}_0} \leftarrow \boldsymbol{s}$
  Execute first action of plan $\boldsymbol{\tau}^0_{\mathbf{a}_0}$

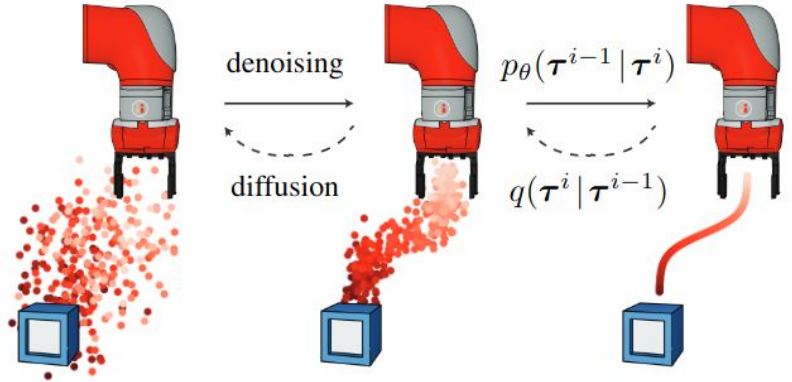
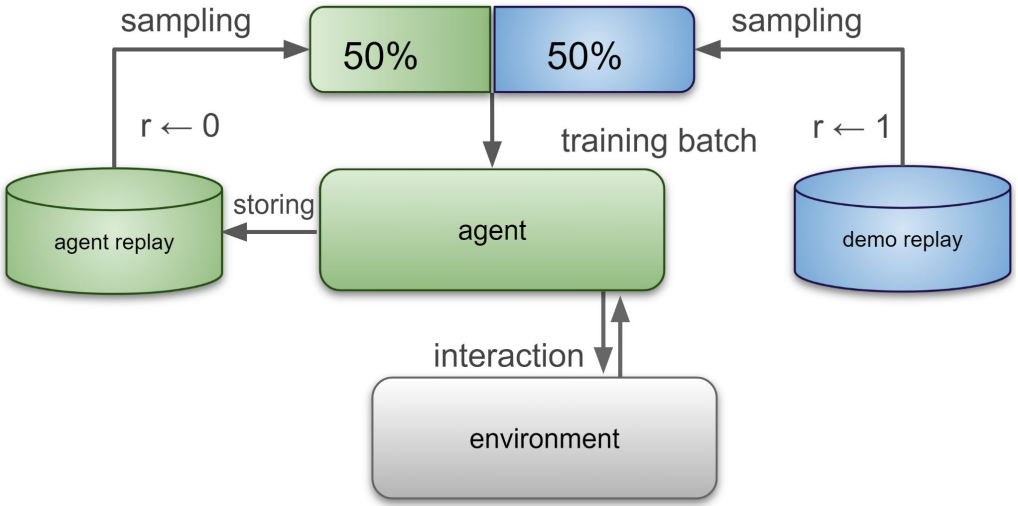


Figure 1. Diffuser is a diffusion probabilistic model that plans by iteratively refining trajectories.

Обучение по экспертному поведению в симуляторе

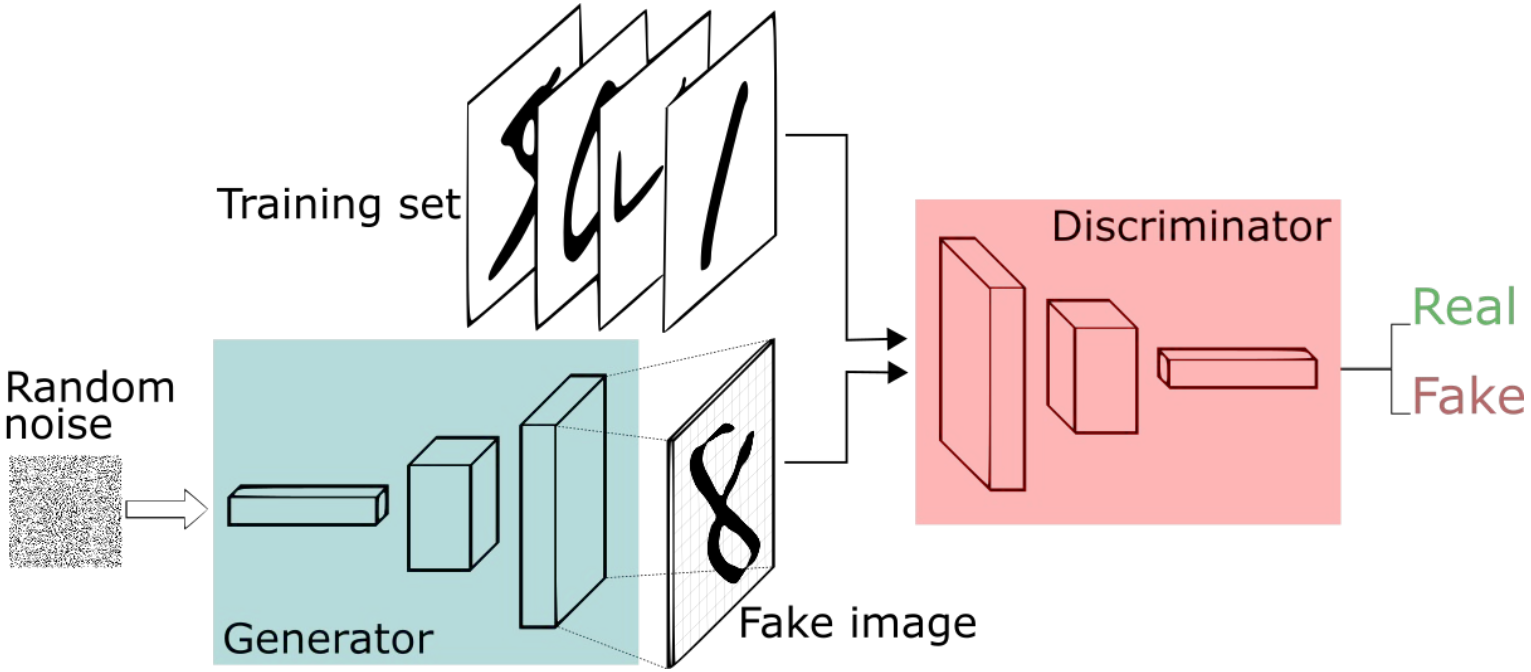
- **SQIL:** Imitation Learning via Reinforcement Learning with Sparse Rewards
- **GAIL:** Generative Adversarial Imitation Learning

Soft-Q Imitation Learning (SQIL):



- первоначально наполняет память прецедентов демонстрациями данными с вознаграждением r = +1;
- при взаимодействии агента со средой и накоплении новых данных добавляет их в память прецедентов с наградой r = 0;
- поддерживает баланс между демонстрационным и новым опытам (50% каждого) в каждом батче

Recap — Generative Adversarial Imitation Learning:



Generative Adversarial Imitation Learning (GAIL):

Algorithm 1 Generative adversarial imitation learning

1: **Input:** Expert trajectories $\tau_E \sim \pi_E$, initial policy and discriminator parameters θ_0, w_0
2: **for** $i = 0, 1, 2, \ldots$ **do**
3: Sample trajectories $\tau_i \sim \pi_{\theta_i}$
4: Update the discriminator parameters from w_i to w_{i+1} with the gradient

обновление дискриминатора →

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_w \log(D_w(s,a))] + \hat{\mathbb{E}}_{\tau_E}[\nabla_w \log(1 - D_w(s,a))] \tag{17}$$

5: Take a policy step from θ_i to θ_{i+1}, using the TRPO rule with cost function $\log(D_{w_{i+1}}(s,a))$. Specifically, take a KL-constrained natural gradient step with

обновление генератора →

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_\theta \log \pi_\theta(a|s) Q(s,a)] - \lambda \nabla_\theta H(\pi_\theta), \quad \text{where } Q(\bar{s},\bar{a}) = \hat{\mathbb{E}}_{\tau_i}[\log(D_{w_{i+1}}(s,a)) \mid s_0 = \bar{s}, a_0 = \bar{a}] \tag{18}$$

6: **end for**

Generative Adversarial Imitation Learning (GAIL):

Algorithm 1 Generative adversarial imitation learning

1: **Input:** Expert trajectories $\tau_E \sim \pi_E$, initial policy and discriminator parameters θ_0, w_0
2: **for** $i = 0, 1, 2, \ldots$ **do**
3: Sample trajectories $\tau_i \sim \pi_{\theta_i}$
4: Update the discriminator parameters from w_i to w_{i+1} with the gradient

D_w – вероятность, что фейк

обновление дискриминатора →

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_w \log(D_w(s,a))] + \hat{\mathbb{E}}_{\tau_E}[\nabla_w \log(1 - D_w(s,a))] \tag{17}$$

5: Take a policy step from θ_i to θ_{i+1}, using the TRPO rule with cost function $\log(D_{w_{i+1}}(s,a))$. Specifically, take a KL-constrained natural gradient step with

обновление генератора →

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_\theta \log \pi_\theta(a|s) Q(s,a)] - \lambda \nabla_\theta H(\pi_\theta),$$
$$\text{where } Q(\bar{s},\bar{a}) = \hat{\mathbb{E}}_{\tau_i}[\log(D_{w_{i+1}}(s,a)) \mid s_0 = \bar{s}, a_0 = \bar{a}] \tag{18}$$

6: **end for**

Generative Adversarial Imitation Learning (GAIL):

Algorithm 1 Generative adversarial imitation learning

1: **Input:** Expert trajectories $\tau_E \sim \pi_E$, initial policy and discriminator parameters θ_0, w_0
2: **for** $i = 0, 1, 2, \ldots$ **do**
3: Sample trajectories $\tau_i \sim \pi_{\theta_i}$
4: Update the discriminator parameters from w_i to w_{i+1} with the gradient

gradient descent

D_w – вероятность, что фейк

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_w \log(D_w(s,a))] + \hat{\mathbb{E}}_{\tau_E}[\nabla_w \log(1 - D_w(s,a))] \tag{17}$$

траектории агента

данные эксперта

5: Take a policy step from θ_i to θ_{i+1}, using the TRPO rule with cost function $\log(D_{w_{i+1}}(s,a))$. Specifically, take a KL-constrained natural gradient step with

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_\theta \log \pi_\theta(a|s) Q(s,a)] - \lambda \nabla_\theta H(\pi_\theta),$$
$$\text{where } Q(\bar{s},\bar{a}) = \hat{\mathbb{E}}_{\tau_i}[\log(D_{w_{i+1}}(s,a)) \mid s_0 = \bar{s}, a_0 = \bar{a}] \tag{18}$$

6: **end for**

Generative Adversarial Imitation Learning (GAIL):

Algorithm 1 Generative adversarial imitation learning

1: **Input:** Expert trajectories $\tau_E \sim \pi_E$, initial policy and discriminator parameters θ_0, w_0
2: **for** $i = 0, 1, 2, \ldots$ **do**
3: Sample trajectories $\tau_i \sim \pi_{\theta_i}$
4: Update the discriminator parameters from w_i to w_{i+1} with the gradient

gradient descent

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_w \log(D_w(s,a))] + \hat{\mathbb{E}}_{\tau_E}[\nabla_w \log(1 - D_w(s,a))] \tag{17}$$

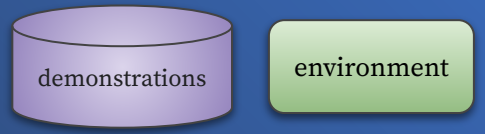
5: Take a policy step from θ_i to θ_{i+1}, using the TRPO rule with cost function $\log(D_{w_{i+1}}(s,a))$. Specifically, take a KL-constrained natural gradient step with

gradient ascent

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_\theta \log \pi_\theta(a|s) Q(s,a)] - \lambda \nabla_\theta H(\pi_\theta), \quad \text{where } Q(\bar{s},\bar{a}) = \hat{\mathbb{E}}_{\tau_i}[\log(D_{w_{i+1}}(s,a)) \mid s_0 = \bar{s}, a_0 = \bar{a}] \tag{18}$$

6: **end for**

Learning from demonstrations with environment



- **DQfD:** Deep Q-learning From Demonstrations
- **R2D3:** Making Efficient Use of Demonstrations to Solve Hard Exploration Problems
- **ForgER:** Forgetful experience replay in hierarchical reinforcement learning from expert demonstrations
- **SMART**: Self-supervised Multi-task Pretraining with Control Transformers

DQfD:

$$r_t + \gamma r_{t+1} + ... + \gamma^{n-1} r_{t+n-1} + max_a \gamma^n Q(s_{t+n}, a)$$

N-Step Return

$$J(Q) = J_{DQ}(Q) + \lambda_1 J_n(Q) + \lambda_2 J_E(Q) + \lambda_3 J_{L2}(Q)$$

L2 regularization

margin

$$J_E(Q) = \max_{a \in A}[Q(s,a) + l(a_E, a)] - Q(s, a_E)$$

Expert margin $l(a_E, a) when a \neq a_E = 0.8$

Algorithm 1 Deep Q-learning from Demonstrations.

Inputs: $\mathcal{D}^{replay}$: initialized with demonstration data set, θ: weights for initial behavior network (random), θ': weights for target network (random), τ: frequency at which to update target net, k: number of pre-training gradient updates
for steps $t \in \{1, 2, \ldots k\}$ **do**
  Sample a mini-batch of n transitions from $\mathcal{D}^{replay}$ with prioritization
  Calculate loss $J(Q)$ using target network
  Perform a gradient descent step to update θ
  if $t \bmod \tau = 0$ **then** $\theta' \leftarrow \theta$ **end if**
end for
for steps $t \in \{1, 2, \ldots\}$ **do**
  Sample action from behavior policy $a \sim \pi^{\epsilon Q_\theta}$
  Play action a and observe (s', r).
  Store (s, a, r, s') into $\mathcal{D}^{replay}$, overwriting oldest self-generated transition if over capacity
  Sample a mini-batch of n transitions from $\mathcal{D}^{replay}$ with prioritization
  Calculate loss $J(Q)$ using target network
  Perform a gradient descent step to update θ
  if $t \bmod \tau = 0$ **then** $\theta' \leftarrow \theta$ **end if**
  $s \leftarrow s'$
end for

R2D3:

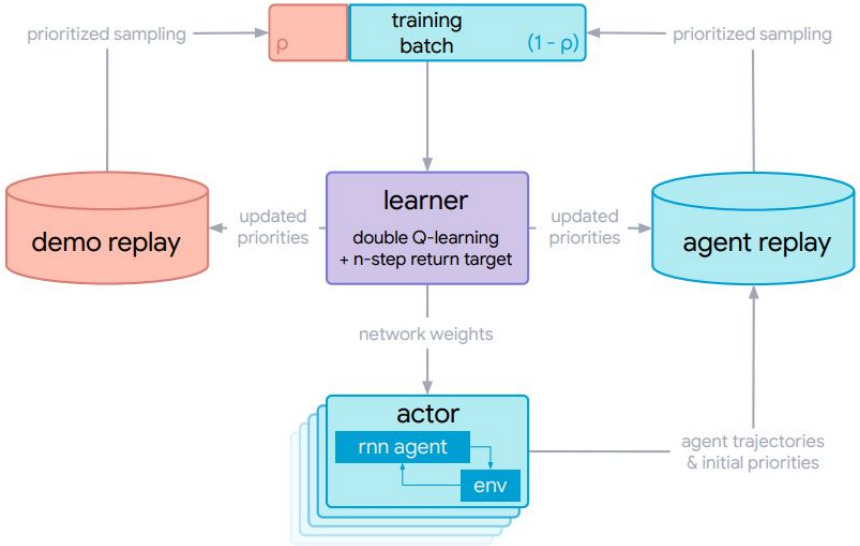
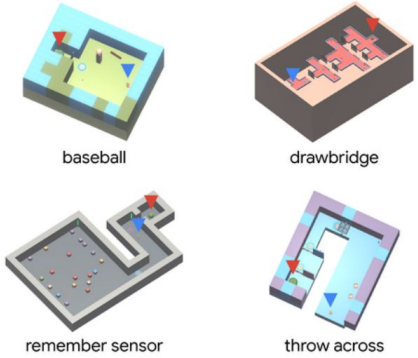


Figure 1 | The R2D3 distributed system diagram. The learner samples batches that are a mixture of demonstrations and the experiences the agent generates by interacting with the environment over the course of training. The ratio between demos and agent experiences is a key hyper-parameter which must be carefully tuned to achieve good performance.



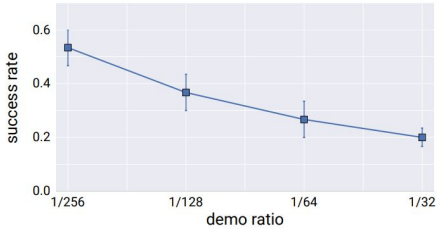
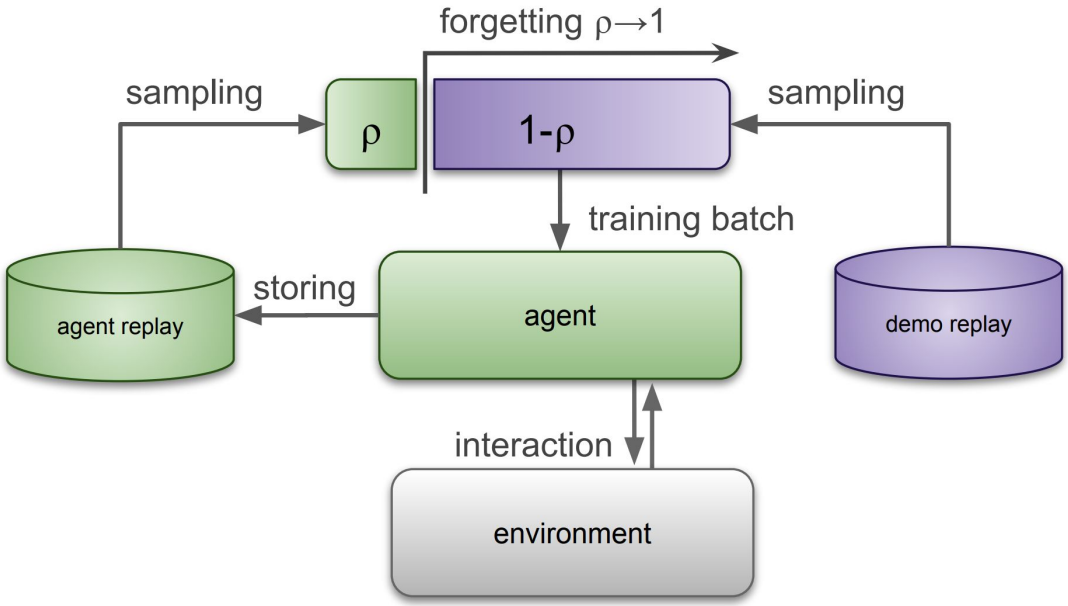
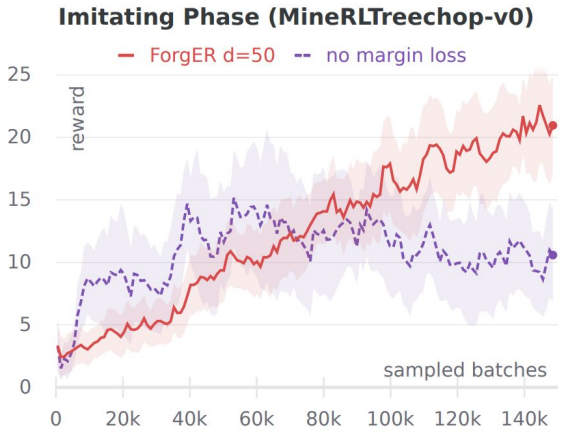
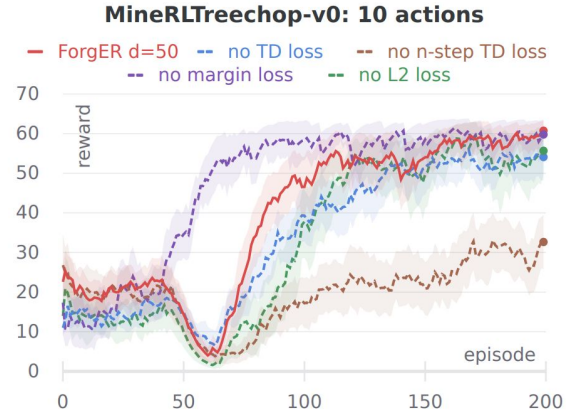


Figure 6 | Success rate (see main text) for R2D3 across all tasks with at least one successful seed, as a function of demo ratio. The square markers for each demo ratio denote the mean success rate, and the error bars show a bootstrapped estimate of the [25,75] percentile interval for the mean estimate. The lower demo ratios consistently outperform the higher demo ratios across the suite of tasks.

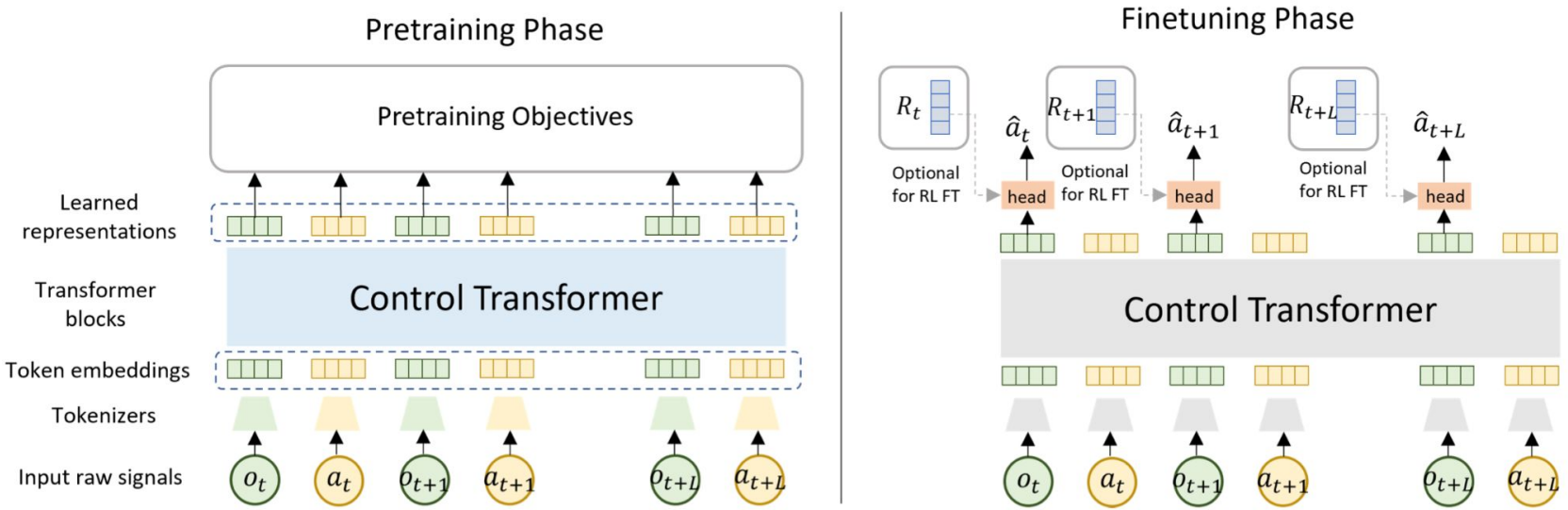
ForgER:


forgetting ρ→1
sampling
ρ
1-ρ
sampling
training batch
agent replay
storing
agent
demo replay
interaction
environment

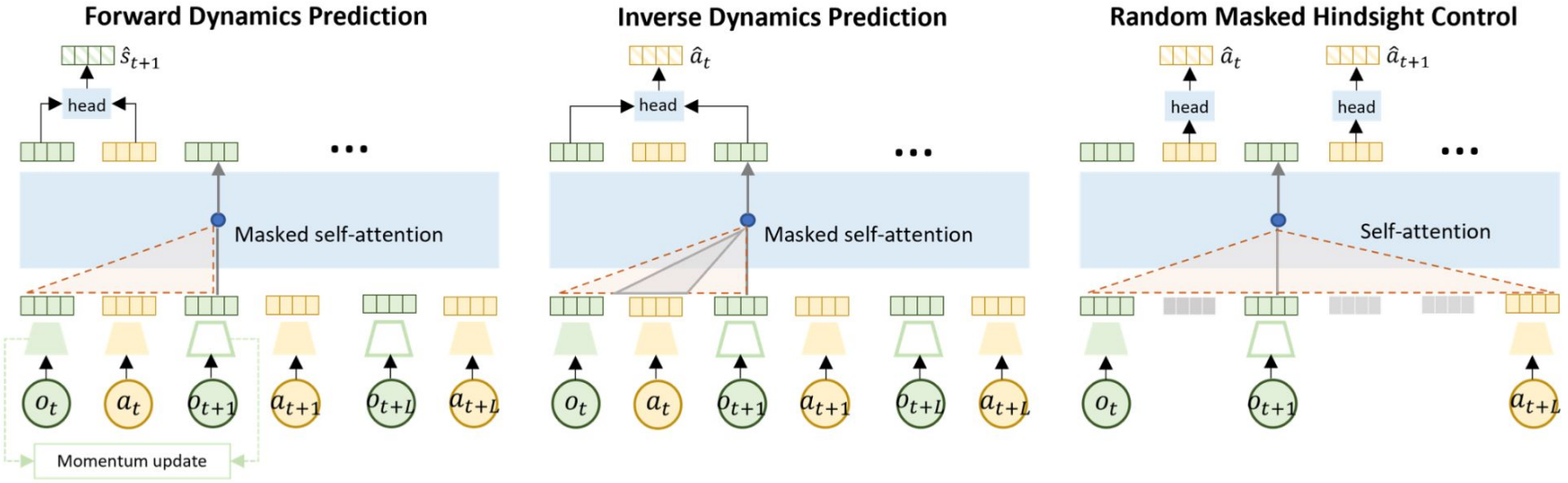

Imitating Phase (MineRLTreechop-v0)
ForgER d=50
no margin loss
reward
sampled batches
0
5
10
15
20
25
0
20k
40k
60k
80k
100k
120k
140k


MineRLTreechop-v0: 10 actions
ForgER d=50
no TD loss
no n-step TD loss
no margin loss
no L2 loss
reward
episode
0
10
20
30
40
50
60
70
0
50
100
150
200

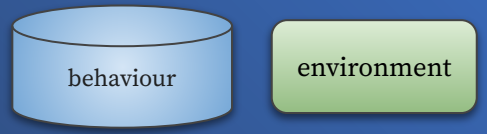
SMART:


Pretraining Phase
Pretraining Objectives
Learned representations
Transformer blocks
Control Transformer
Token embeddings
Tokenizers
Input raw signals
o_t
a_t
o_{t+1}
a_{t+1}
o_{t+L}
a_{t+L}
Finetuning Phase
R_t
R_{t+1}
R_{t+L}
$\hat{a}_t$
$\hat{a}_{t+1}$
$\hat{a}_{t+L}$
Optional for RL FT
Optional for RL FT
Optional for RL FT
head
head
head
Control Transformer
o_t
a_t
o_{t+1}
a_{t+1}
o_{t+L}
a_{t+L}

SMART:

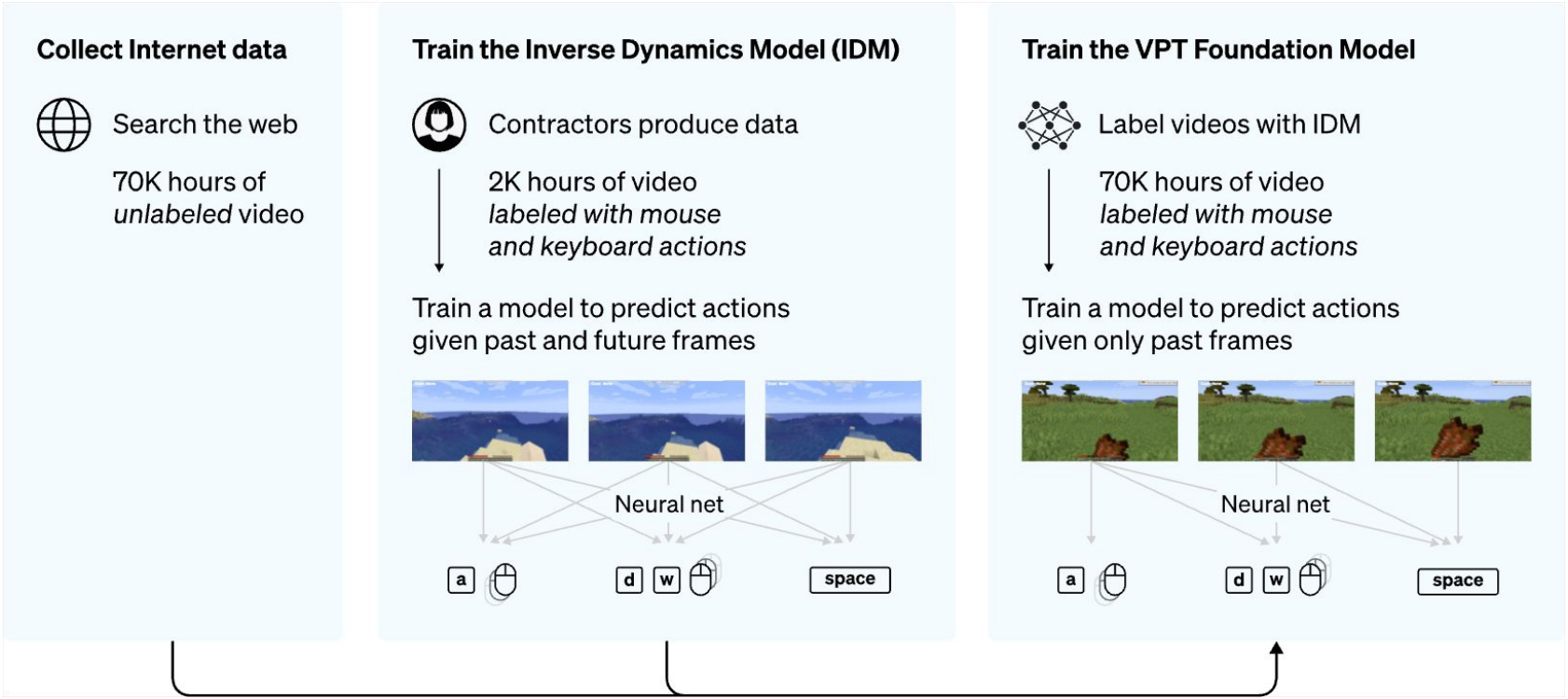

Forward Dynamics Prediction
$\hat{s}_{t+1}$
head
Masked self-attention
o_t
a_t
o_{t+1}
a_{t+1}
o_{t+l}
a_{t+L}
Momentum update
Inverse Dynamics Prediction
$\hat{a}_t$
head
Masked self-attention
o_t
a_t
o_{t+1}
a_{t+1}
o_{t+l}
a_{t+L}
Random Masked Hindsight Control
$\hat{a}_t$
$\hat{a}_{t+1}$
head
head
Self-attention
o_t
o_{t+1}
a_{t+L}

Learning from behaviour with environment



- **VPT:** Video PreTraining: Learning to Act by Watching Unlabeled Online Videos
- **AlphaStar:** Grandmaster level in StarCraft II using multi-agent reinforcement learning

VPT: Video PreTraining



VPT:

$$\min_{\theta} \sum_{t \in [1 \ldots T]} -\log \pi_\theta(a_t | o_1, \ldots, o_t), \text{ where } a_t \sim p_{\text{IDM}}(a_t | o_1, \ldots, o_t, \ldots, o_T)$$

~9 дней на 720 V100 GPUs

VPT:

One must choose between using a shared network or separate networks to represent the policy and value function. Using separate networks avoids interference between objectives, while using a shared network allows useful features to be shared. PPG is able to achieve the best of both worlds by splitting optimization into two phases, one that advances training and one that distills features. Compared to PPO, PPG significantly improves sample efficiency on the challenging Procgen Benchmark.

Algorithm 1 PPG

for phase $= 1, 2, \ldots$ **do**
  Initialize empty buffer B
  for iteration $= 1, 2, \ldots, N_\pi$ **do** ▷ Policy Phase
    Perform rollouts under current policy π
    Compute value function target $\hat{V}_t^{\text{targ}}$ for each state s_t
    for epoch $= 1, 2, \ldots, E_\pi$ **do** ▷ Policy Epochs
      Optimize $L^{clip} + \beta_S S[\pi]$ wrt θ_π
    for epoch $= 1, 2, \ldots, E_V$ **do** ▷ Value Epochs
      Optimize L^{value} wrt θ_V
    Add all $(s_t, \hat{V}_t^{\text{targ}})$ to B
  Compute and store current policy $\pi_{\theta_{old}}(\cdot|s_t)$ for all states s_t in B
  for epoch $= 1, 2, \ldots, E_{aux}$ **do** ▷ Auxiliary Phase
    Optimize L^{joint} wrt θ_π, on all data in B
    Optimize L^{value} wrt θ_V, on all data in B

for epoch $= 1, 2, \ldots, E_\pi$ **do**
  Optimize $L^{clip} + \beta_S S[\pi]$ wrt θ_π

entropy bonus

$$L^{clip} = \hat{\mathbb{E}}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t) \right]$$

$$L^{value} = \hat{\mathbb{E}}_t \left[\frac{1}{2}(V_{\theta_V}(s_t) - \hat{V}_t^{\text{targ}})^2 \right]$$

$$L^{joint} = L^{aux} + \beta_{clone} \cdot \hat{\mathbb{E}}_t \left[KL[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)] \right]$$

$$L^{aux} = \frac{1}{2} \cdot \hat{\mathbb{E}}_t \left[(V_{\theta_\pi}(s_t) - \hat{V}_t^{\text{targ}})^2 \right]$$

VPT:

Чтобы предотвратить катастрофическое забывание навыков предварительно обученной сети в процессе дообучения, подход включает в себя дополнительную функцию потерь (KL loss) между моделью RL и замороженной предварительно обученной стратегией.

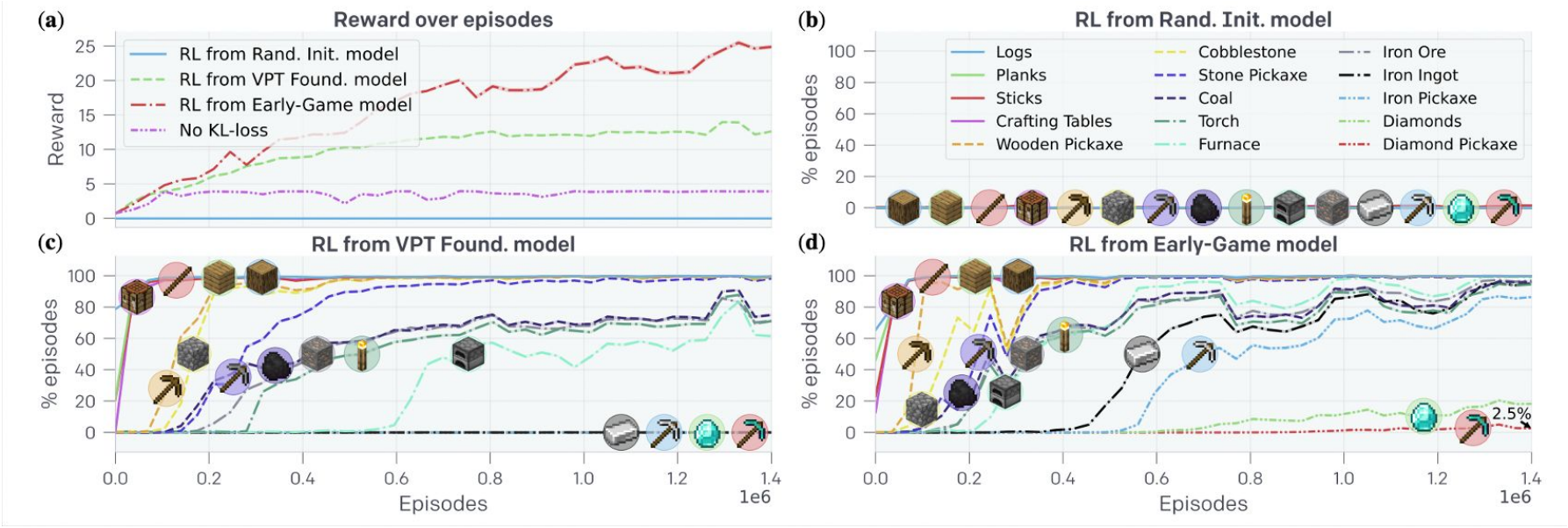
$$L_{klpt} = \rho \mathrm{KL}(\pi_{pt}, \pi_\theta)$$

entropy loss is replaced with the KL loss between the policy being trained and the pretrained one

Algorithm 1 PPG

for phase $= 1, 2, \ldots$ **do**
  Initialize empty buffer B
  for iteration $= 1, 2, \ldots, N_\pi$ **do** ▷ Policy Phase
    Perform rollouts under current policy π
    Compute value function target $\hat{V}_t^{\text{targ}}$ for each state s_t
    for epoch $= 1, 2, \ldots, E_\pi$ **do** ▷ Policy Epochs
      Optimize $L^{clip} + \beta_S S[\pi]$ wrt θ_π
    for epoch $= 1, 2, \ldots, E_V$ **do** ▷ Value Epochs
      Optimize L^{value} wrt θ_V
    Add all $(s_t, \hat{V}_t^{\text{targ}})$ to B
  Compute and store current policy $\pi_{\theta_{old}}(\cdot|s_t)$ for all states s_t in B
  for epoch $= 1, 2, \ldots, E_{aux}$ **do** ▷ Auxiliary Phase
    Optimize L^{joint} wrt θ_π, on all data in B
    Optimize L^{value} wrt θ_V, on all data in B

VPT:

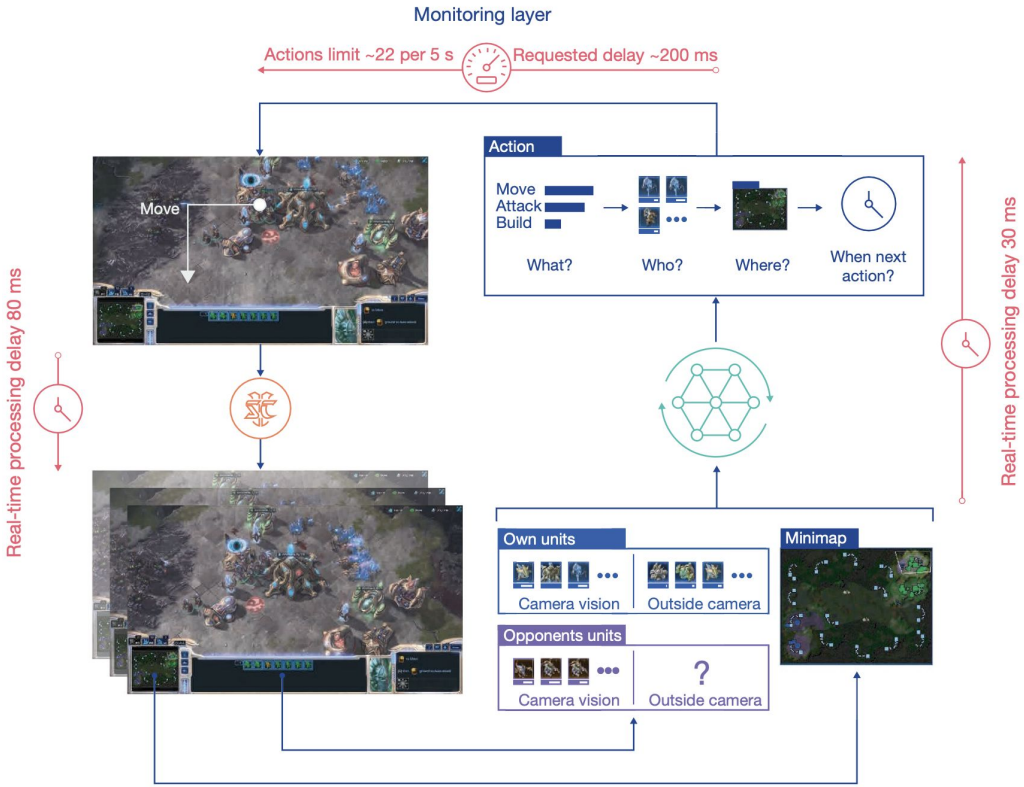

(a)
Reward over episodes
Reward
RL from Rand. Init. model
RL from VPT Found. model
RL from Early-Game model
No KL-loss
(b)
RL from Rand. Init. model
% episodes
Logs
Planks
Sticks
Crafting Tables
Wooden Pickaxe
Cobblestone
Stone Pickaxe
Coal
Torch
Furnace
Iron Ore
Iron Ingot
Iron Pickaxe
Diamonds
Diamond Pickaxe
(c)
RL from VPT Found. model
% episodes
Episodes
1e6
(d)
RL from Early-Game model
% episodes
2.5%
Episodes
1e6

VPT:

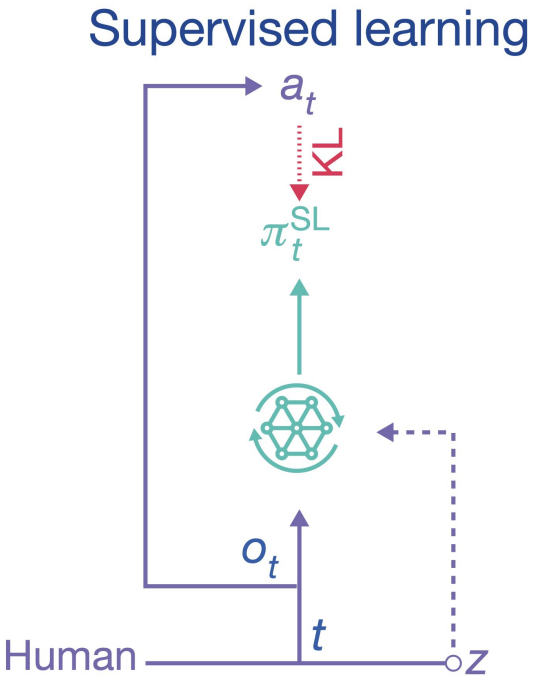
Figure 16: Items obtained when RL fine-tuning from the early-game model without a KL loss. The model learns to obtain all items that the early-game model can craft zero-shot, which are logs, planks, sticks, and a crafting table. In contrast to the treatment with a KL-penalty, it does not learn any items beyond these initial four, likely because skills that are not performed zero-shot, and for which the model thus does not initially see any reward, are catastrophically forgotten while the first four items are learned.

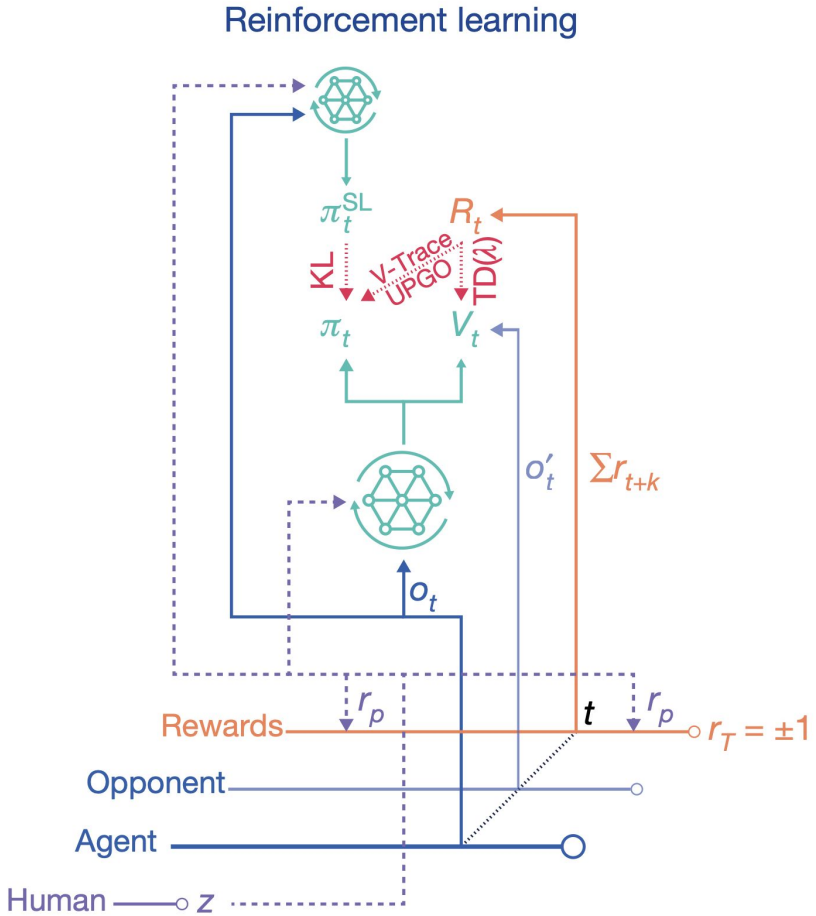
AlphaStar:

Наблюдение AlphaStar – мини-карта и список юнитов

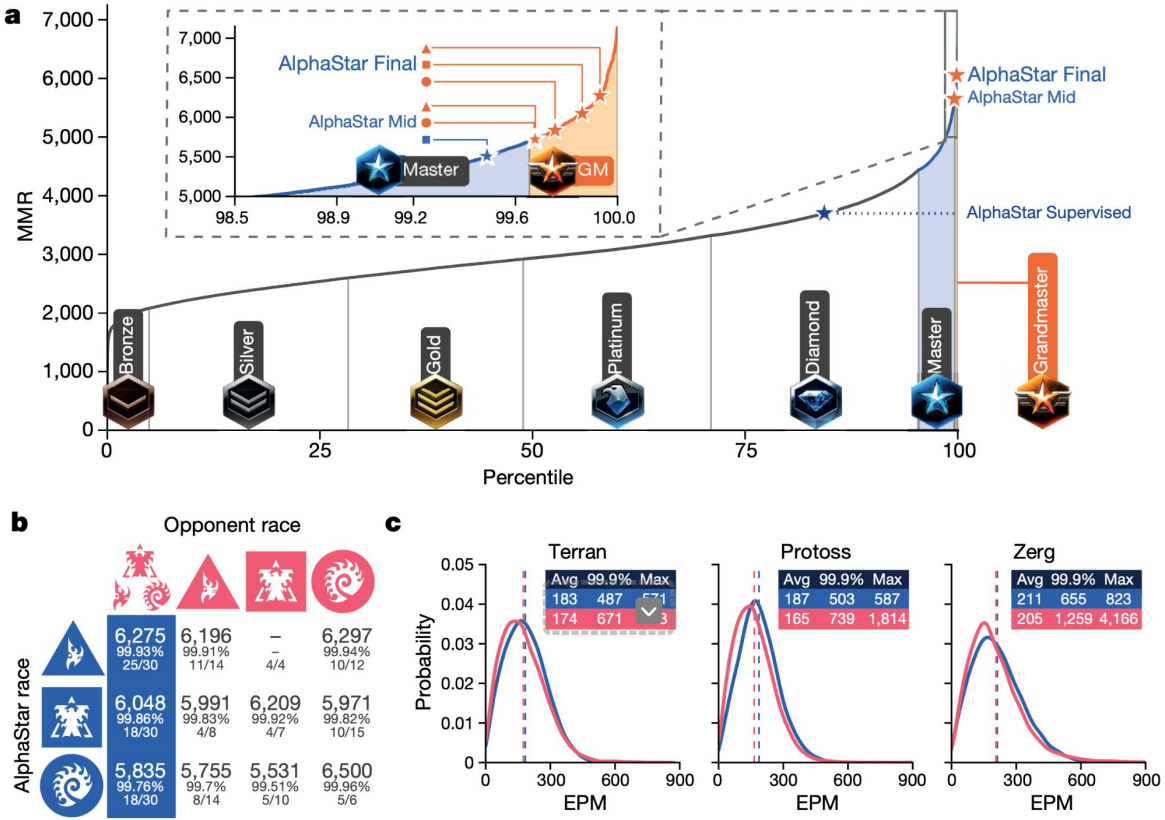


AlphaStar:


Supervised learning
a_t
KL
π_t^{SL}
o_t
t
Human
z


Reinforcement learning
π_t^{SL}
R_t
KL
V-Trace
UPGO
TD(λ)
π_t
V_t
o'_t
Σr_{t+k}
o_t
r_p
t
r_p
Rewards
$r_T = \pm 1$
Opponent
Agent
Human
z

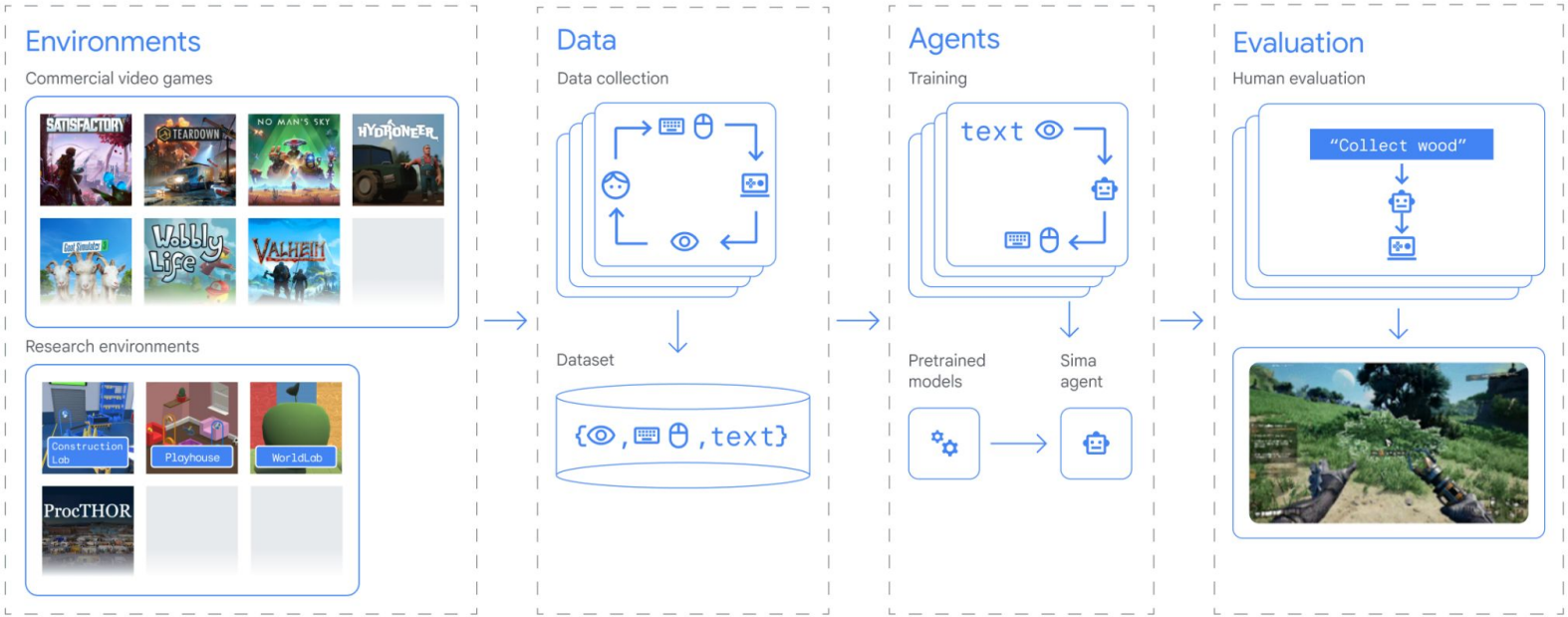
AlphaStar:


a
MMR
Percentile
AlphaStar Final
AlphaStar Mid
AlphaStar Supervised
Bronze
Silver
Gold
Platinum
Diamond
Master
Grandmaster
Master
GM
b
Opponent race
AlphaStar race
6,275 99.93% 25/30
6,196 99.91% 11/14
– – 4/4
6,297 99.94% 10/12
6,048 99.86% 18/30
5,991 99.83% 4/8
6,209 99.92% 4/7
5,971 99.82% 10/15
5,835 99.76% 18/30
5,755 99.7% 8/14
5,531 99.51% 5/10
6,500 99.96% 5/6
c
Terran
Protoss
Zerg
Avg 99.9% Max
183 487
174 671
187 503 587
165 739 1,814
211 655 823
205 1,259 4,166
Probability
EPM

Спасибо за внимание!

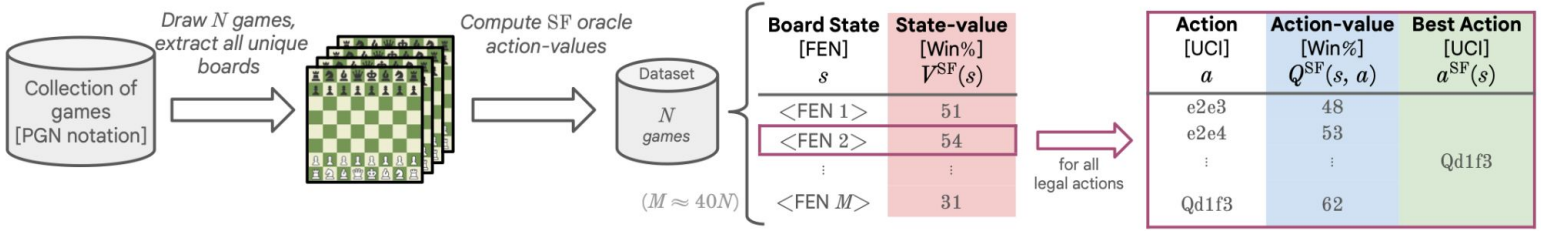
Дополнительно: недавние работы*

A generalist AI agent for 3D virtual environments



https://deepmind.google/discover/blog/sima-generalist-ai-agent-for-3d-virtual-environments/

Grandmaster-Level Chess Without Search:



Datasets

Training sets

Source: games from Lichess (range of Elo)

10k 1M 10M

1k

Test set

14.7% overlap with training set (mostly early-game boards)

Source: curated chess puzzles from Lichess

10k

Puzzle Test set

1.33% overlap with training set

Predictors

State-value

Loss: $-\log P_\theta^{SV}(V^{SF}(s)|s)$

Policy:

$$\hat{a}(s) = \arg\min_{a \in \mathcal{A}_{legal}(s)} \hat{V}_\theta(\underbrace{T(s,a)}_{s'})$$

$$\hat{V}_\theta(s') := \mathbb{E}_{P_\theta^{SV}(z|s')}[z]$$

Legal action leading to next state with minimal predicted expected state-value for opponent player.

Action-value

Loss: $-\log P_\theta^{AV}(Q^{SF}(s,a)|s,a)$

Policy:

$$\hat{a}(s) = \arg\max_{a \in \mathcal{A}_{legal}(s)} \hat{Q}_\theta(s,a)$$

$$\hat{Q}_\theta(s,a) := \mathbb{E}_{P_\theta^{AV}(z|s,a)}[z]$$

Legal action with maximal predicted expected action-value.

Behavioral cloning

Loss: $-\log P_\theta^{BC}(a^{SF}(s)|s)$

Policy:

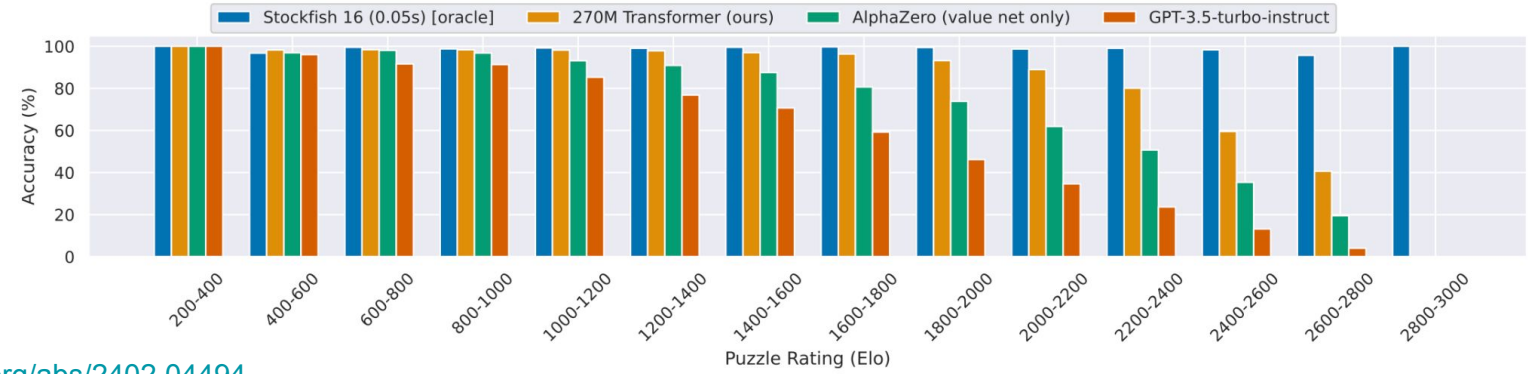
$$\hat{a}(s) = \arg\max_{a \in \mathcal{A}_{legal}(s)} P_\theta^{BC}(a|s)$$

Legal action with highest predicted probability.

https://arxiv.org/abs/2402.04494

Grandmaster-Level Chess Without Search:

Agent	Search	Input	Tournament Elo	Lichess Elo		Accuracy (%)		Kendall's τ
				vs. Bots	vs. Humans	Puzzles	Actions	
9M Transformer (ours)		FEN	2007 (±15)	2054	-	85.5	64.2	0.269
136M Transformer (ours)		FEN	2224 (±14)	2156	-	92.1	68.5	0.295
270M Transformer (ours)		FEN	**2299** (±14)	**2299**	**2895**	**93.5**	**69.4**	**0.300**
GPT-3.5-turbo-instruct		PGN	-	1755	-	66.5	-	-
AlphaZero (policy net only)		PGN	1620 (±22)	-	-	61.0	-	-
AlphaZero (value net only)		PGN	1853 (±16)	-	-	82.1	-	-
AlphaZero (400 MCTS simulations)	✓	PGN	2502 (±15)	-	-	95.8	-	-
Stockfish 16 (0.05s) [oracle]	✓	FEN	2706 (±20)	2713	-	99.1	100.0	1.000



https://arxiv.org/abs/2402.04494

Спасибо за внимание!